# GCF: A Token-Optimized Wire Format for Structured LLM Interactions

dayna@blackwell-systems.com · DOI: 10.5281/zenodo.20579817

Dayna Blackwell, Blackwell Systems

2026-07-02 (v7)

## Abstract

AI agents consume and produce structured data under fixed token budgets. The dominant encoding is JSON, which fails at scale for two compounding reasons. First, overhead: a tokenization breakdown of 500-row JSON payloads across 8 tokenizers shows 52.4% of tokens are repeated field names and 28.6% are structural characters, leaving only 19% for actual data values (Section 1.1). Second, ambiguity: JSON's grammar characters merge with adjacent content in BPE tokenizer vocabularies, hiding structural boundaries inside single tokens. A cross-tokenizer analysis of 43 tokenizers from 20 providers shows this is universal: JSON's combined adversarial surface spans 1,939 mergeable words. Controlled experiments across two architectures (GPT-NeoX, Llama), two scales (410M, 1.3B), and three domains (structured data, code, molecular notation) prove that preventing delimiter merges produces 3-738x better structured data comprehension, 1.5x better code comprehension, and zero natural language cost. Every attention head in a standard BPE model is structurally constrained by delimiter merging; the model has 4x more structural attention capacity than the tokenizer allows it to use. JSON's structural choices create both the overhead that wastes tokens and the ambiguity that wastes attention.

We present GCF, a bidirectional text-based wire format designed to eliminate both problems. GCF supports two encoding profiles: a **graph profile** exploiting referential identity (local IDs), topological encoding (edge arrows), and hierarchical grouping (section headers); and a **generic profile** encoding arbitrary structured data with positional rows, pipe separators, and inline primitive arrays. GCF's grammar characters were selected from the set empirically verified to have near-zero merge rates across all 43 tested tokenizers: pipe (0.47% merge rate, 24 mergeable words) vs JSON's quote (8.17%, 193 words) vs TOON's tab (32.91%, 1,238 words).

We evaluated GCF across 2,500+ LLM evaluations spanning 11 models and 4 providers (Anthropic, OpenAI, Google, Mistral). No model has been trained on GCF.

**Comprehension (generic profile):** 27 runs across 11 models, 500-order nested data. GCF achieves **100%** on every frontier model (Opus, Sonnet, Haiku, GPT-5.5, Gemini 2.5 Pro, Gemini 3.1 Pro, Gemini 3.5 Flash). TOON is the weakest format.

**Comprehension (graph profile):** 25 runs across 10 models, 500 symbols with 200 edges. GCF averages 90.7% accuracy where TOON averages 68.8% and JSON averages 54.1%. GCF wins 24 of 25 runs (1 tie, 0 losses).

**Generation:** 28 runs across 9 models. GCF achieves 5/5 validity on every frontier model. TOON's official decoder rejects LLM-generated output on 7 of 9 models due to a structural design flaw in flat tabular encoding. GCF output is 63% smaller than JSON and 33% smaller than TOON.

**Token efficiency:** GCF achieves 50-92% fewer tokens than JSON depending on data complexity and session reuse. 50-69% on a single call (generic through graph profile). Up to 92% with session deduplication across repeated tool calls. On TOON's own benchmark (their datasets, their tokenizer, their methodology), expanded from 6 to 16 datasets, GCF wins 15 of 16 (29% fewer tokens overall).

**Lossless verification:** 43 billion+ round-trips across 5 formats (JSON, YAML, MessagePack, CSV, TOML) with zero failures. Validated across 17 serialization formats in the Format Mega-Gauntlet.

Session deduplication (84.3% cumulative savings over a 5-call session) and delta encoding (81.2% on re-queries) compound savings across multi-turn interactions. A streaming encoding extension enables zero-buffering encode with O(1) memory per row. The format is implemented in six languages (Go, TypeScript, Python, Rust, Swift, Kotlin), 157 conformance fixtures, and deployed in production MCP servers. Specification v3.2 Stable: gcformat.com.

A companion paper, "Tokenizer-Attention Coupling: How BPE Merge Decisions Permanently Shape Transformer Internal Organization" DOI: 10.5281/zenodo.20925910, presents the full mechanistic analysis: controlled experiments across two architectures and two scales proving that BPE merge decisions permanently constrain which attention heads develop. Key findings: every attention head in a standard BPE model is structurally stranded (384/384 at 410M, 768/768 at 1.3B show 4x more delimiter attention when given clean boundaries), the damage is immediate (present by step 5,000) and permanent (unchanged through step 40,000), and at 1.3B scale standard BPE develops 124 counterproductive delimiter heads whose removal improves comprehension by 57%. The mechanism generalizes across structured data (3-738x), code (1.5x), and molecular notation (2.2x).

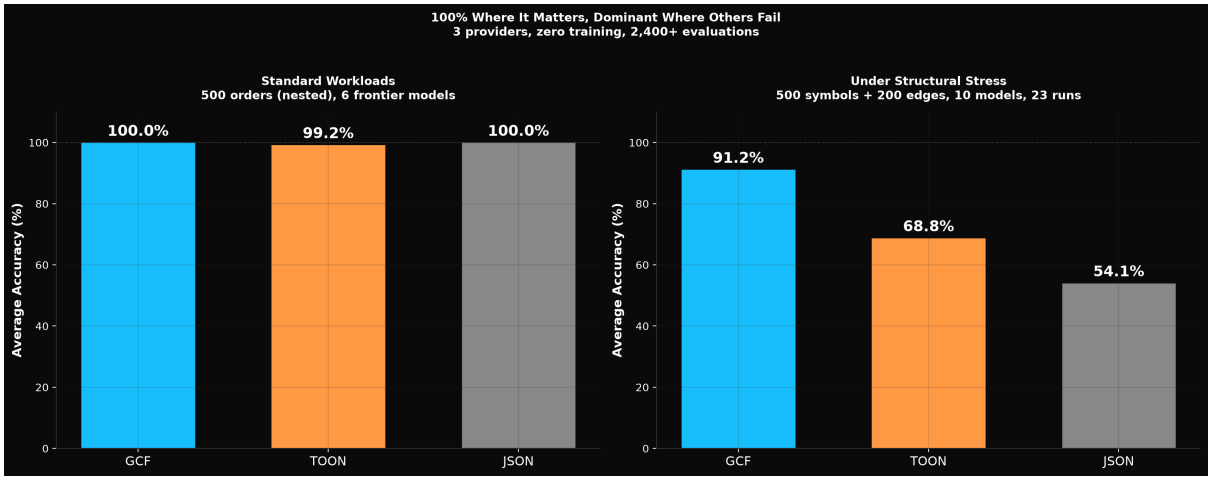


**Figure 1:** Comprehension and Generation across 11 models and 4 providers

---

## 1. Why JSON Fails at Scale

The Model Context Protocol (MCP) defines how AI agents interact with external tools. Tool responses are overwhelmingly encoded as JSON. This is convenient for developers but expensive for the consumer that matters most: the language model itself.

JSON fails at scale for two compounding reasons, not one.

### 1.1 The Overhead Problem

Consider a typical MCP tool response returning 10 graph nodes and 8 edges (a blast radius query, a dependency subgraph, or a context retrieval result). In JSON, this payload consumes 965 tokens. The

same semantic content in GCF consumes 233 tokens. The difference (732 tokens, 75.9% reduction) is pure waste: field name repetition, structural delimiters (`{`, `}`, `[`, `]`, `:`, `","`), and full qualified names repeated in every edge reference.

At 500 rows, 81% of JSON's tokens are overhead. Only 19% carry actual data values. This waste compounds across a task. An agent making 5 tool calls during a code change task consumes ~4,825 tokens on JSON tool responses. In GCF, the same 5 calls consume ~1,165 tokens, and less with session statefulness enabled. The difference, ~3,660 tokens, is context window capacity that could hold source code, documentation, or additional tool results.

### 1.2 The Structural Ambiguity Problem

Token overhead is the visible problem. The deeper problem is invisible: JSON's grammar characters merge with adjacent content in BPE tokenizer vocabularies, hiding structural boundaries inside single tokens.

When GPT-4's tokenizer encounters `"name":"Alice"`, it produces `["name][":"][Alice]["]` (4 tokens). The opening quote has fused with the field name into token #32586. Claude's tokenizer produces `["][name][":"][Alice]["]` (5 tokens). The structural boundary (where the field name starts) is at a different token position depending on which model processes the data.

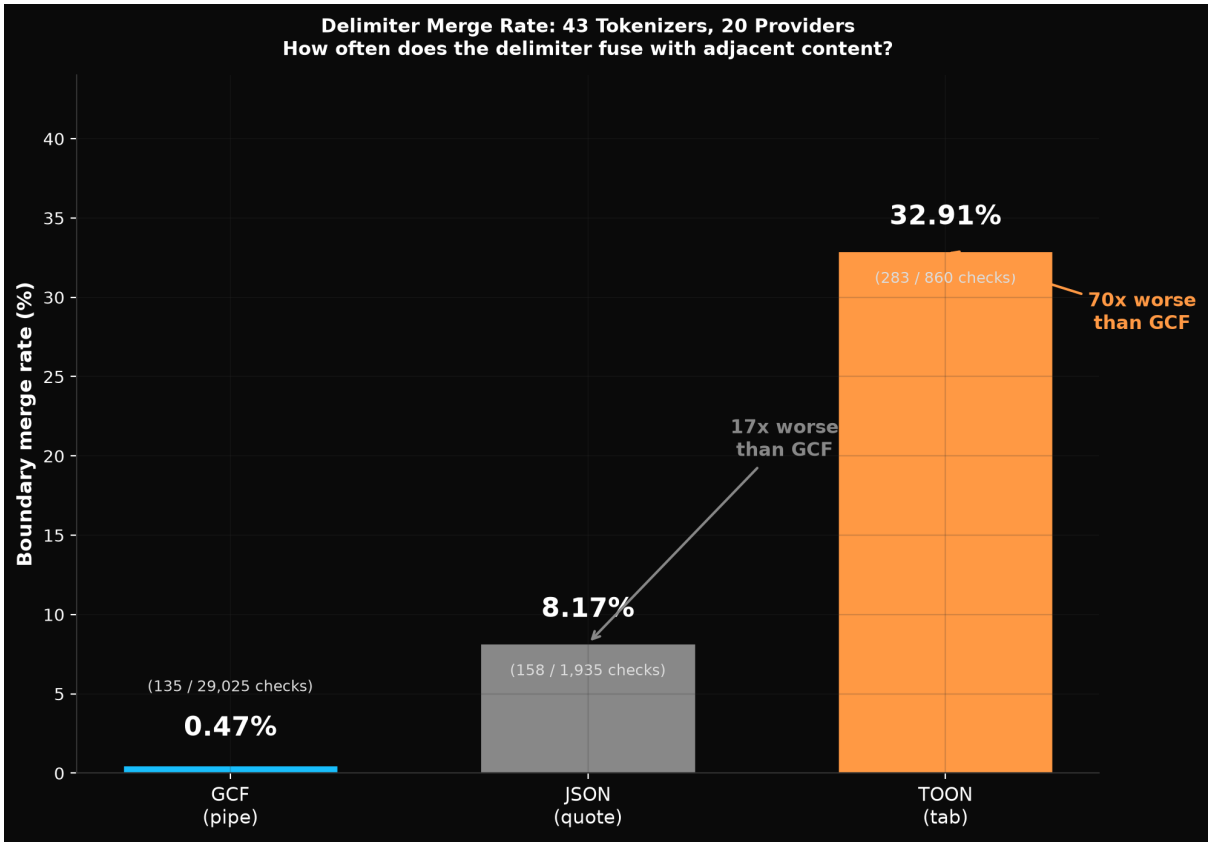


**Figure 2:** Delimiter merge rates: pipe 0.47%, quote 8.17%, tab 32.91%

An exhaustive vocabulary scan across 43 tokenizers from 20 providers reveals that this is universal:

- JSON's opening quote merges with common field names (`id`, `name`, `type`, `value`, `url`) on **30% of tokenizers**
- JSON's combined adversarial surface across all 7 grammar characters: **1,939 mergeable words**
- TOON's tab delimiter: **1,238 mergeable words** (GPT-4o has a 100% tab merge rate)
- GCF's pipe delimiter: **24 mergeable words** (all TypeScript union keywords, none are field names)

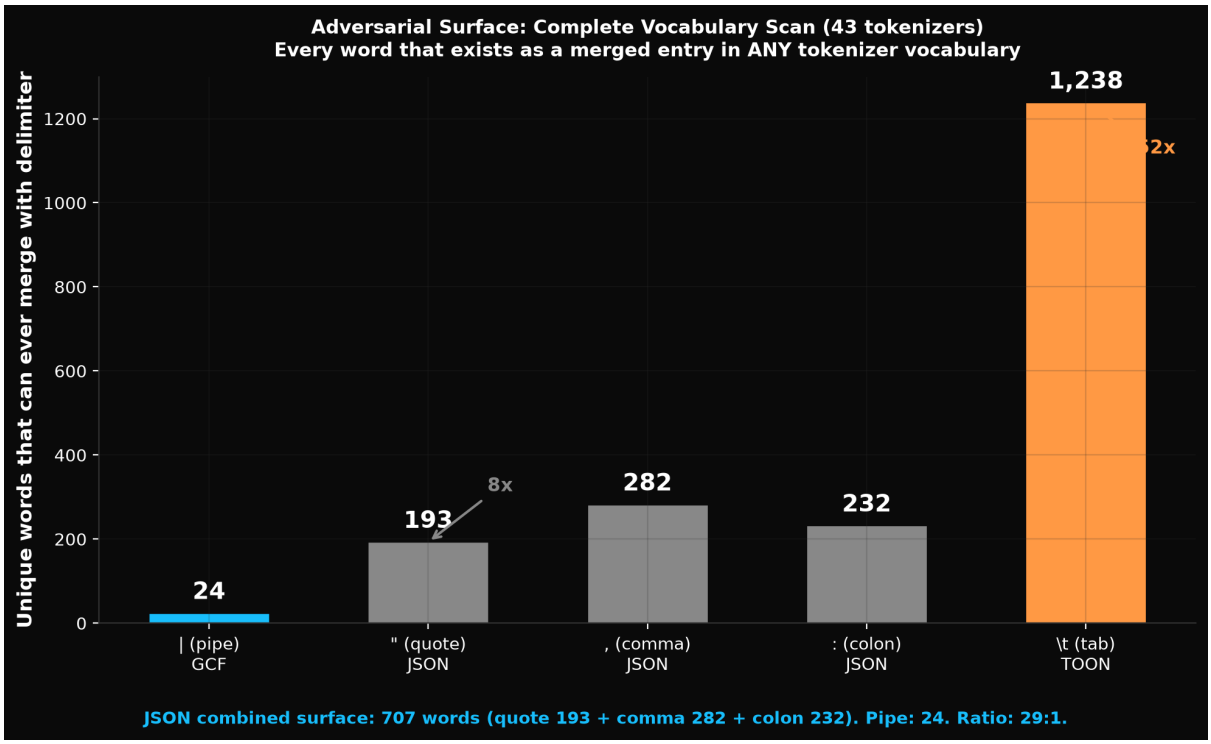


**Figure 3:** Vocabulary merge entries by tokenizer

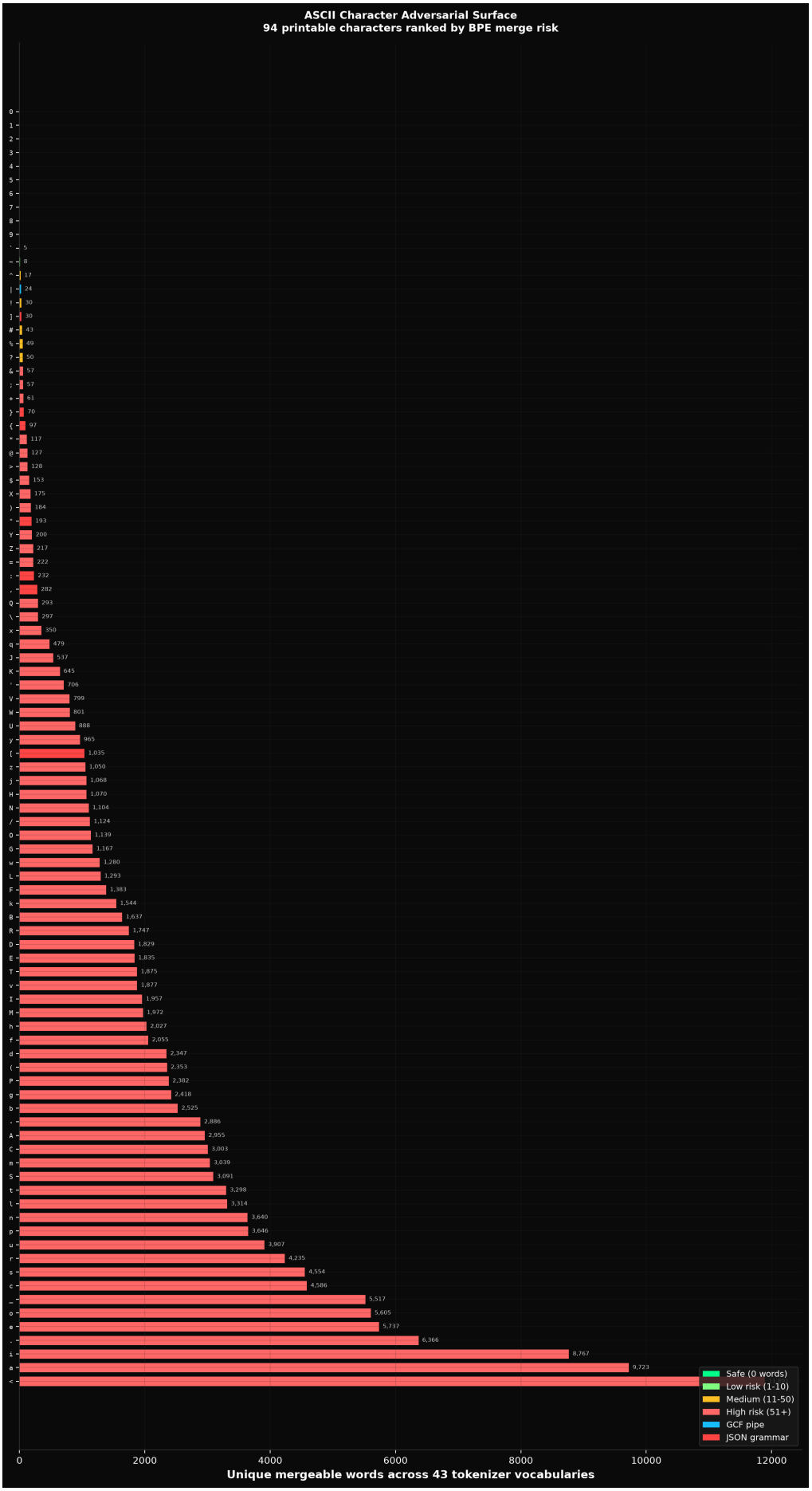


**Figure 4:** ASCII adversarial surface: all 94 printable characters ranked by merge risk

At 500 rows, the overhead problem and the ambiguity problem compound. The model must process approximately 8,500 overhead tokens (81% of the sequence) that are also structurally ambiguous (field boundaries hidden inside merged tokens). Production model probing (Mistral 7B, Llama 3.1 8B) shows attention entropy crosses the comparison format at 50 rows and grammar attention collapses from 30% to 8.6%. The model stops tracking structure entirely.

Controlled experiments prove this is causal. Models trained with merge barriers (16 delimiter characters forbidden from BPE merges) achieve 3-738x lower structured data perplexity, 1.5x lower code perplexity, and develop 50-161 concentrated structural attention heads, with zero natural language cost (19.4 vs 19.5). At 1.3B scale, every attention head in the standard model shows 4x more delimiter attention when simply given clean token boundaries, revealing that BPE permanently constrains the model's structural capacity. The full analysis, including stranded heads, scaling validation, and domain generalization across code and molecular notation, is in the companion paper, "Tokenizer-Attention Coupling" DOI: 10.5281/zenodo.20925910.

GCF was designed to eliminate both problems: header factoring eliminates the overhead, and delimiter selection from the empirically verified low-merge character set eliminates the ambiguity. To our knowledge, GCF is the only wire format whose grammar was designed from the model's perspective rather than the developer's (Section 2.1).

### 1.3 Why This Matters Now

Three trends make this problem urgent:

1. **Tool-heavy agent workflows.** Modern AI agents make 10-50 tool calls per task. Each call returns JSON. Token budgets are consumed by tool overhead before the agent finishes its work.
2. **Graph-structured responses are growing.** Code intelligence, dependency analysis, knowledge graphs, and system topology queries all return graph data. Graph payloads are JSON's worst case because they contain repeated node references across edges.
3. **Context window costs are real.** Whether measured in dollars (API billing), latency (time-to-first-token), or capability (what fits in the window), every wasted token reduces the agent's effectiveness.

### 1.4 Why Not Just Use Binary?

Binary formats (Protocol Buffers, MessagePack, FlatBuffers) optimize for byte size and decode speed. But LLMs consume text, not bytes. A protobuf-encoded tool response must be decoded to text before the LLM can process it, and the decoded text is typically JSON, eliminating the savings at the point of consumption.

The optimization target is not bytes on wire. It is tokens in the context window. These are different quantities with different solutions.

---

## 2. Design Principles

GCF's design starts from a question no prior format asked: which characters can the attention mechanism actually use as structural landmarks?

### 2.1 Tokenizer-Informed Delimiter Selection

Every wire format must choose delimiter characters. JSON chose `"`, `:`, `,`, `{`, `}`. TOON chose tab. CSV chose comma. These choices were made based on human readability or data engineering conventions, not on how language models process them.

GCF chose pipe (`|`) for field separation and `@` for identity references based on empirical analysis of 43 BPE tokenizer vocabularies from 20 providers. The selection criteria: a delimiter must be its own token on every tested tokenizer, must have near-zero merge rate with adjacent content, and must have minimal adversarial surface (few vocabulary entries where it fuses with common words).

| Delimiter | Merge rate | Mergeable words | Used by |
|---|---|---|---|
| Pipe `\|` | 0.47% | 24 (all TypeScript unions) | GCF |
| Quote `"` | 8.17% | 193 | JSON |
| Tab `\t` | 32.91% | 1,238 | TOON |

The companion paper ("Tokenizer-Attention Coupling") proves this choice is causally important. When delimiter characters merge with content, every attention head in the model loses structural capacity: 384/384 heads at 410M and 768/768 at 1.3B show 4x more delimiter attention when given clean boundaries. At 1.3B, standard BPE models develop 124 counterproductive delimiter heads whose removal improves comprehension by 57%. The delimiter is not a cosmetic choice. It determines whether the model's attention mechanism can parse structure.

GCF's remaining design principles build on this foundation. Once delimiters are clean, three encoding strategies reduce overhead.

### 2.2 Referential Identity

Graph nodes are referenced multiple times: once in their declaration and once per edge they participate in. In JSON, each reference repeats the full qualified name:

```
{
  "source": "github.com/org/repo/internal/mcp.NewServer",
  "target": "github.com/org/repo/internal/mcp.requireHash",
  "edge_type": "calls"
}
```

In GCF, nodes are declared once with a local ID and referenced by that ID thereafter:

```
@0 fn github.com/org/repo/internal/mcp.requireHash 0.78 lsp_resolved
@4 fn github.com/org/repo/internal/mcp.NewServer 0.54 lsp_resolved
@0<@4 calls
```

The full qualified name appears once per node. Every subsequent reference is 2-3 tokens (`@0`, `@4`) instead of 15-20 tokens.

### 2.3 Topological Encoding

JSON encodes edges as objects with named fields. GCF encodes edges as directed connections:

**JSON (1 edge, ~12 tokens):**

```
{"source": "...", "target": "...", "edge_type": "calls"}
```

**GCF (1 edge, ~5 tokens):**

```
@0<@4 calls
```

The `<` arrow encodes direction. The source and target are local IDs. The edge type is a bare token. No field names, no delimiters, no quoting.

### 2.4 Hierarchical Grouping

Graph query results often partition nodes by distance from a query center: direct targets (distance 0), related symbols (distance 1), extended context (distance 2+). In JSON, each node carries a `"distance": N` field. In GCF, a section header replaces all per-node distance fields:

```
## targets
@0 fn ... 0.78 lsp_resolved
@1 method ... 0.74 lsp_resolved
## related
@4 fn ... 0.54 lsp_resolved
```

One header replaces N repeated fields.

---

## 3. Specification

### 3.1 Grammar

```
payload       = header LF { section } [ summary ] ;
section       = group-header LF { line LF } ;
```

```
line          = node-line | edge-line | ref-line | tabular-row
              | kv-line | nested-ref | inline-array | comment ;
summary       = "##! summary" SP key-value { SP key-value } LF ;

header        = "GCF" SP key-value { SP key-value } ;
group-header  = "##" SP group-name [ SP "[" count-or-deferred "]" [ field-
decl ] ] ;
count-or-deferred = count | "?" ;
field-decl    = "{" field-name { "," field-name } "}" ;
node-line     = "@" id SP kind SP qname SP score SP provenance ;
edge-line     = "@" target "<" "@" source SP edge-type [ SP status ] ;
ref-line      = "@" id SP SP "# previously transmitted" ;
tabular-row   = [ "@" id SP ] value { "|" value } ;
kv-line       = key "=" value ;
inline-array  = key "[" count-or-deferred "]" ":" SP value { "," value } ;
nested-ref    = "." field-name ;
comment       = "#" SP text ;

id            = DIGIT { DIGIT } ;
count         = DIGIT { DIGIT } ;
kind          = "fn" | "type" | "method" | "iface" | "var" | "const"
              | "resource" | "table" | "class" | "selector" | "field"
              | "route" | "ext" | "file" | "pkg" | "svc" ;
status        = "added" | "removed" ;
```

### 3.2 Header

The header line identifies the format and carries payload metadata:

```
GCF profile=graph tool=context_for_task budget=5000 tokens=1847 symbols=10 edges=8
```

`tool` identifies the MCP tool that produced this response. It is optional: recommended for MCP workloads, omitted for non-MCP applications. Making `tool` optional decouples the graph profile from MCP-specific usage, so the same encoding works in standalone code intelligence tools, data pipelines, or any context where graph-structured data enters an LLM without an MCP server in the loop. `budget` and `tokens` enable the consumer to assess utilization. `symbols` and `edges` give counts without scanning.

### 3.3 Node Lines

```
@{id} {kind} {qualified_name} {score} {provenance}
```

Fields are positional. No field names, no delimiters beyond whitespace. Kind abbreviations reduce token count (`fn` vs `"function"`, `iface` vs `"interface"`).

| Abbreviation | Full form |
|---|---|
| `fn` | function |
| `type` | type |
| `method` | method |
| `iface` | interface |
| `var` | var |
| `const` | const |
| `resource` | resource |
| `table` | table |
| `class` | class |
| `selector` | selector |
| `field` | field |
| `route` | route_handler |
| `ext` | external |
| `file` | file |
| `pkg` | package |
| `svc` | service |

### 3.4 Edge Lines

```
@{target}<@{source} {edge_type} [{status}]
```

The `<` arrow points toward the target. `@0<@4 calls` means "@4 calls @0." Status is optional: `added` or `removed` for diff payloads.

### 3.5 Group Headers

```
## targets
## related
## extended
## edges [N]
```

Group headers partition the payload into semantic sections. The group a node appears in encodes its distance from the query center, eliminating per-node distance fields. The edges section header includes `[N]` (the edge count) to enable direct count verification by the LLM without scanning.

### 3.6 Generic Profile (Generic Encoding)

The graph profile (Sections 3.1-3.5) encodes typed nodes and edges. The generic profile encodes arbitrary structured data using the same grammar primitives.

**Tabular arrays:**

```
## {name} [{count}]{{field1},{field2},{field3}}
value1|value2|value3
```

The header declares field names once. Rows are pipe-separated positional values. No field names repeated per record.

```
## employees [3]{id,name,department,salary}
1|Alice Smith|Engineering|95000
2|Bob Jones|Sales|72000
3|Carol Wu|Marketing|85000
```

**Primitive arrays** (all elements are scalars) are inlined on a single line:

```
tags[3]: production,us-east-1,critical
ports[3]: 8080,8443,9090
```

**Key-value pairs** for primitive object fields:

```
config=production
port=5432
active=true
```

**Section headers** for nested objects:

```
## database
  host=db.example.com
  port=5432
```

**Nested fields in tabular rows** use `@{id}` prefixes and `.field {}`:

```
## orders [2]{id,total,status,customer}
@0 1001|249.99|shipped|^
  .customer {}
    name=Alice Smith
    tier=premium
```

**Nested object flattening** eliminates attachments entirely when a nested object has the same keys in every row and all leaf values are scalars. The nested keys become path columns in the tabular header using > as the path separator:

```
## orders [500]{id,total,status,customer>name,customer>tier}
1001|249.99|shipped|Alice Smith|premium
1002|89.50|pending|Bob Chen|standard
```

At 500 rows, this eliminates 500 attachment blocks (1,000+ lines of `@{id}` markers, `.field {}` headers, and `key=value` pairs) and replaces them with inline pipe-separated values. The LLM sees a single flat table with positional columns. Flattening is applied recursively: nested objects within nested objects flatten to multi-level paths (`address>city>zip`). Fields containing the > character in their names are excluded from flattening and fall back to attachments, preserving the invariant that > in a column name always indicates a path.

**Value encoding rules:**

| Type | Encoding |
|---|---|
| String | bare text |
| Number | unquoted decimal |
| Boolean | lowercase `true`/`false` |
| Null | – |
| Empty string | `""` |
| String containing \| or \n | quoted, with \" and \\ escaping |

**Uniformity detection:** An array is tabular-eligible if the first 5 elements are objects with at least 70% key overlap with the first element, accommodating semi-uniform data.

### 3.7 Session Statefulness

Across multiple tool calls in a session, previously-transmitted nodes can be referenced without re-transmission:

```
GCF profile=graph tool=context_for_files tokens=800 symbols=5 edges=1 session=true
## targets
@0  # previously transmitted
@7 fn github.com/org/repo/internal/mcp.handleBlastRadius 0.62 lsp_resolved
## edges [1]
@0<@7 calls
```

The `session=true` header flag enables ID persistence. A bare `@0` (no kind, name, or score) references a node transmitted in a previous response. Multi-call workflows get progressively cheaper as the session builds a shared vocabulary.

This exploits a property unique to agent tool interactions: the consumer (the LLM) maintains conversational state across calls. A traditional wire format cannot assume its consumer remembers previous messages. GCF can because LLM context windows are, by definition, stateful.

### 3.8 Streaming Encoding Extension

When the encoder does not know payload size upfront (data arriving incrementally from a database cursor, API pagination, or graph traversal), it uses streaming mode:

```
GCF profile=graph tool=context_for_task budget=5000
## targets
@0 fn pkg.Auth 0.95 lsp_resolved
@1 fn pkg.Handler 0.88 lsp_resolved
## related
@2 type pkg.Config 0.72 ast_inferred
## edges [?]
@0<@1 calls
@2<@0 references
##! summary symbols=3 edges=2 sections=targets:2,related:1,edges:2
```

The `[?]` deferred count marker signals that the count will be provided in the trailer. The `##! summary` line provides all counts after the data is complete. The LLM has both the data and the counts in its context window (recency bias in transformer attention means the trailer is at least as strong a signal as header counts).

Streaming mode enables zero-buffering encode: rows emit the instant they are produced, with O(1) memory per row. This is critical for MCP servers that walk large graphs or paginate results; the LLM starts receiving context immediately instead of waiting for the full traversal.

TOON cannot add streaming without a breaking spec change (their grammar mandates upfront `[N]` with no deferred count or trailer mechanism).

---

## 4. Implementation Status

GCF is not a speculative format proposal. It is implemented in six languages, published to seven package registries, covered by 157 conformance fixtures, verified across 43 billion+ lossless round-trips, and deployed in production MCP servers.

The implementation includes:

- **Go library** (`github.com/blackwell-systems/gcf-go`, v0.6.1): Encode, Decode, EncodeGeneric, DecodeGeneric, EncodeWithSession, EncodeDelta, StreamEncoder, GenericStreamEncoder. Zero dependencies.
- **TypeScript library** (`@blackwell-systems/gcf` on npm, v0.6.1): encode, decode, encodeGeneric, decodeGeneric, encodeWithSession, encodeDelta, StreamEncoder, GenericStreamEncoder. Zero dependencies, ESM.
- **Python library** (`gcf-python` on PyPI, v0.5.1): encode, decode, encode_generic, decode_generic, encode_with_session, encode_delta, StreamEncoder, GenericStreamEncoder. Zero dependencies, Python 3.9+.
- **Rust library** (`gcf` on crates.io, v0.5.1): encode, decode, encode_generic, decode_generic, encode_with_session, encode_delta, StreamEncoder, GenericStreamEncoder. Minimal dependencies (serde_json).
- **Swift library** (`gcf-swift` via SPM, v0.5.1): encode, decode, encodeGeneric, decodeGeneric, encodeWithSession, encodeDelta, StreamEncoder, GenericStreamEncoder. Zero dependencies.
- **Kotlin library** (`gcf-kotlin` via JitPack, v0.5.1): encode, decode, encodeGeneric, decodeGeneric, encodeWithSession, encodeDelta, StreamEncoder, GenericStreamEncoder. Zero dependencies.
- **MCP proxy** (`github.com/blackwell-systems/gcf-proxy`): drop-in wrapper for any MCP server, re-encodes JSON responses as GCF with streaming progress notifications. Zero code changes to upstream.
- **Conformance test suite** (157 v3 fixtures across both profiles): language-agnostic JSON fixtures validating encode, decode, session, delta, generic, streaming, and normative error cases.
- **Specification** (SPEC.md, v3.2 Stable): RFC 2119 keywords, conformance checklists, decoder error taxonomy, streaming extension, security considerations. Published at gcformat.com.

#### Correctness Validation

All six implementations are tested against round-trip invariants and the shared conformance suite. A graph response encoded as GCF and decoded back must preserve node identity, kind, score, provenance, group membership, edge direction, edge type, and optional status metadata. The generic profile is validated against 18 additional fixtures covering flat arrays, nested objects, value formatting, primitive array inlining, and edge cases.

#### Production Deployment

GCF is deployed across 12 production systems spanning infrastructure, network automation, code intelligence, API tooling, and developer tools:

- **OmniRoute** (6.1K stars): AI gateway and proxy. GCF vendored into headroom compression engine, replacing custom encoder. 55-62% savings, 100% payload coverage. TOON rejected (npm dependency). github.com/diegosouzapw/OmniRoute

- **NetClaw** (556 stars): AI network automation, 113 skills, 66 MCP integrations. Replaced TOON entirely after benchmarking. 55.8% savings vs JSON, 13.6% fewer tokens than TOON. github.com/automateyournetwork/netclaw
- **NeuroNest**: Agent-first IDE by Network Guardian. First independent commercial adoption. GCF across 4 encoding surfaces with session dedup, delta encoding, per-provider comprehension gate, and shadow mode A/B testing. neuronest.cc
- **ctx** (510 stars): Claude Code context optimizer. GCF for recommendation bundles, graph queries, wiki search. 51.5% savings, up to 57.8% on bundles. github.com/stevesolun/ctx
- **Speakeasy**: API tooling for OpenAPI ecosystem. GCF as native output format (`oq --format gcf`). Merged after dependency audit confirming no external data transmission. github.com/speakeasy-api/openapi
- **Open Data Products SDK** (Linux Foundation): GCF sidecars for ODPC/ODPG workflows. opendataproducts.org/sdk
- **bb** (Bitbucket Cloud CLI): GCF as opt-in output format for agent consumers. Independent Go adoption. github.com/payfacto/bb
- **knowing** (28 MCP tools): GCF as primary output format for code intelligence, with session deduplication and delta encoding. 84% token savings per tool call.
- **agent-lsp** (66 MCP tools): GCF tabular output via EncodeGeneric for symbol lists, references, diagnostics, and call hierarchies. 34-44% savings.
- **Gladys** (7.5K stars): Home automation. PR replacing TOON with GCF for MCP tool responses. 40% fewer tokens, 36% fewer than TOON. github.com/GladysAssistant/Gladys
- **Raycast**: JSON-to-GCF converter extension, published to Raycast Store.
- **SkyElite**: Multi-agent travel planner. GCF for inter-agent context sharing. 3rd place, National AI Hackathon (Pakistan, 2026).

Full adoption details with benchmarks: gcformat.com/ecosystem/adopters

---

## 5. Where Token Savings Come From

GCF's token savings compound across three layers: per-call structural efficiency, session-level deduplication, and delta encoding on re-queries.

### 5.1 Graph Profile Savings (per call)

The graph profile achieves 75-80% savings through five mechanisms:

| Source | JSON cost | GCF cost | Savings per occurrence |
|---|---|---|---|
| Field names | 9 field names x ~2 tokens each = ~18 tokens/symbol | 0 (positional) | ~18 tokens/symbol |
| Edge references | 2 qualified names x ~15 tokens = ~30 tokens/edge | 2 local IDs x ~1 token = ~2 tokens/edge | ~28 tokens/edge |
| Structural delimiters | `{`, `}`, `[`, `]`, `:`, `","` = ~6 tokens/symbol | 0 | ~6 tokens/symbol |
| Distance fields | `"distance": N` = ~3 tokens/symbol | 0 (implicit in group) | ~3 tokens/symbol |
| Kind strings | `"function"` = ~2 tokens | `fn` = 1 token | ~1 token/symbol |

For a 10-symbol, 8-edge payload: JSON ~965 tokens, GCF ~233 tokens. The 732-token difference breaks down roughly as: 280 from field name elimination, 224 from edge reference compression, 60 from delimiter removal, 30 from group headers, and the remainder from kind abbreviations and whitespace. Savings increase with payload size because edge references (the largest per-item saving) grow proportionally.

#### 5.2 Generic Profile Savings (per call)

The generic profile achieves 50-69% savings:

| Source | JSON cost | GCF cost | Savings per occurrence |
|---|---|---|---|
| Field names | repeated per record | declared once in header | ~(N-1) x fields per array |
| Structural delimiters | `{`, `}`, `:`, `,`, `"` per record | `\|` between values | ~6 tokens/record |
| Array framing | `[`, `]`, commas | `[count]` in header | fixed |
| Primitive arrays | `["a","b","c"]` with brackets and quotes | `name[3]: a,b,c` | ~50% per array |
| Nested object flattening | braces + field names per row | `>` path columns in header | eliminates all nesting overhead for uniform objects |

| Source | JSON cost | GCF cost | Savings per occurrence |
|---|---|---|---|
| Non-uniform nesting | braces + field names | `.field {}` + `key=value` or `^{fields}` inline | ~50% per nested object |

**Nested object flattening (v3.2)** is the largest single optimization for nested data. When a nested object has the same keys in every row (e.g., `address` with `street`, `city`, `zip`), those keys become path columns (`address>street`, `address>city`, `address>zip`) in the tabular header. The nested object disappears entirely from the row encoding: no attachment blocks, no extra indentation, no repeated field names. At 500 rows with a 3-field nested object, this eliminates 1,500 field name repetitions and 500 attachment headers. The data becomes a single flat table that the LLM reads positionally.

For 2,000 employee records with 6 fields: JSON ~127,050 tokens, GCF ~49,055 tokens (61% savings). Across 16 real-world datasets representing actual MCP tool responses: 29% fewer tokens than the next most efficient format, 56% fewer than JSON.

### 5.3 Session Deduplication (multi-call)

In agentic workflows, the same tool is called repeatedly across a conversation. Session deduplication eliminates retransmission of previously-seen nodes by referencing their local IDs from prior responses.

| Call | Without dedup | With dedup | Cumulative savings |
|---|---|---|---|
| 1st | 233 tokens | 233 tokens | 0% |
| 2nd | 233 tokens | 154 tokens | 34% |
| 3rd | 233 tokens | 89 tokens | 62% |
| 4th | 233 tokens | 47 tokens | 80% |
| 5th | 233 tokens | 28 tokens | 84% |

By the fifth tool call in a session, GCF transmits 88% fewer tokens than the first call because most symbols have already been declared. JSON has no equivalent mechanism: every response repeats every field name, every qualified name, every structural delimiter from scratch. Session dedup compounds with per-call savings: the 76% per-call saving becomes 97% by the fifth call relative to JSON.

This requires local IDs (`@0`, `@1`, …) and a session header (`session=true`). Formats without local ID systems cannot implement session dedup without a fundamental redesign.

### 5.4 Delta Encoding (re-queries)

When a user re-asks a question or queries overlapping data, delta encoding transmits only what changed:

```
GCF profile=graph tool=context_for_task tokens=120 delta=true
## targets [2]
@14 fn pkg/new.HandleNew 0.91 lsp_resolved
@15 type pkg/new.Config 0.88 ast_inferred
## edges [1]
@14<@3 calls
```

Instead of retransmitting the full 233-token payload, the delta sends only new or changed nodes. Measured savings on re-queries: **81.2%** beyond the already-compressed GCF encoding. Combined with session dedup, a fifth re-query of overlapping data can transmit under 10 tokens where JSON would transmit 965.

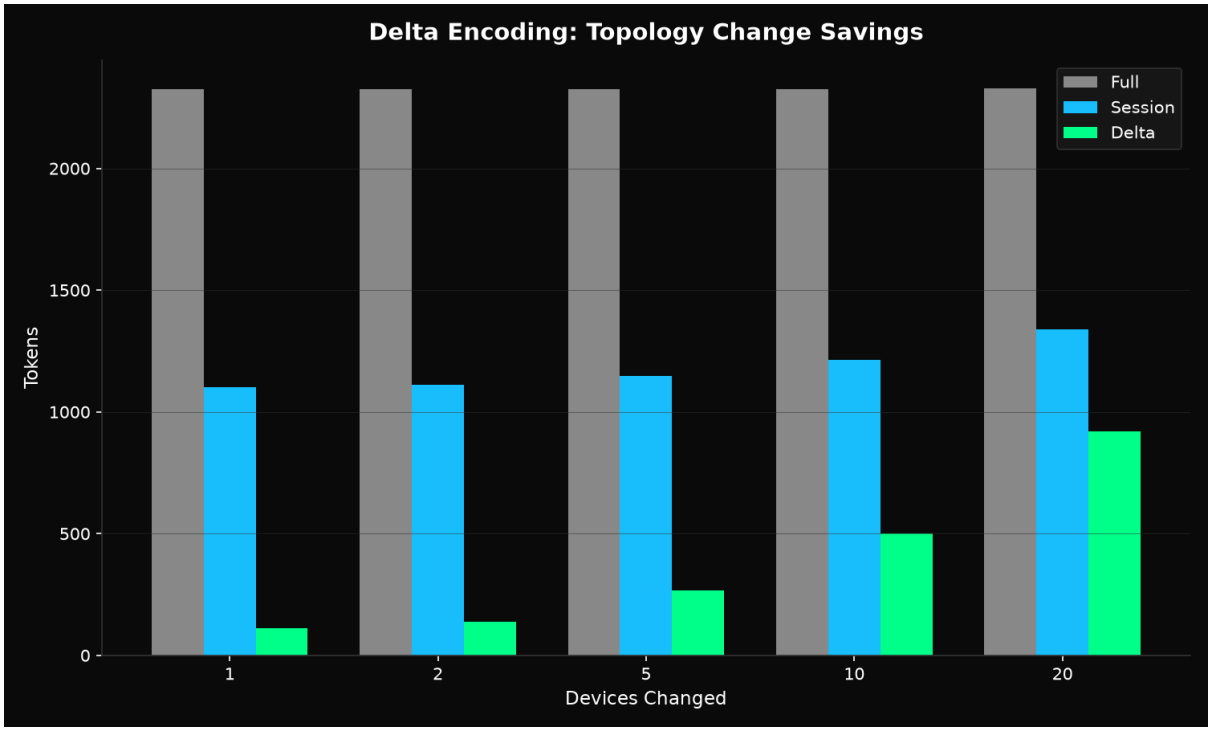


**Figure 5:** Delta topology savings

### 5.5 Total Compounding

The three layers compound multiplicatively:

| Layer | Savings vs JSON | Mechanism |
|---|---|---|
| Per-call encoding | 50-80% | Header factoring, positional values, edge compression, flattening |
| Session deduplication | +34-88% of remainder | Local ID references to previously-transmitted nodes |
| Delta encoding | +81% of remainder | Transmit only changes |

At the fifth tool call with overlapping data: JSON transmits ~965 tokens. GCF transmits ~10-28 tokens. The cumulative saving exceeds 97%. No competing format offers all three layers because session dedup and delta encoding require local IDs, which require a format designed for multi-turn interactions rather than single-shot serialization.

---

## 6. Benchmarks

All benchmarks encode the same semantic content in JSON and GCF. Token counts in Section 6.1 use cl100k_base; the 500-symbol eval (Section 6.1b) and TOON comparison (Section 7.5) use o200k_base (matching TOON's benchmark methodology). Measurements taken on Apple M4 Pro.

### 6.1 Token Comparison

| Payload | Symbols | Edges | JSON tokens | GCF tokens | Savings |
|---|---|---|---|---|---|
| context_for_task | 10 | 8 | 965 | 233 | 75.9% |
| context_for_task (large) | 30 | 24 | 2,968 | 649 | 78.1% |
| context_for_files | 15 | 12 | 1,490 | 334 | 77.6% |
| blast_radius | 8 | 6 | 835 | 208 | 75.1% |
| semantic_diff | 12 | 10 | 1,206 | 295 | 75.5% |
| graph_query | 20 | 16 | 2,078 | 423 | 79.6% |

**Median token savings: 76.7%**

Savings increase with payload size because the ratio of edge references to node declarations grows, and edge references are where compression is strongest.

### 6.1a Generic Profile: Standard Workloads

27 runs across 11 models and 4 providers. 500 orders with nested customer objects and line items. 13 structured extraction questions. Zero format instructions. Deterministic answers, no LLM judge.

| Model | Provider | Runs | GCF avg | TOON avg | JSON avg |
|---|---|---|---|---|---|
| Claude Opus 4.6 | Anthropic | 2 | **100%** | 100% | 100% |
| Claude Sonnet 4.6 | Anthropic | 3 | **100%** | 97.4% | 100% |
| Claude Haiku 4.5 | Anthropic | 4 | **100%** | 100% | 100% |
| GPT-5.5 | OpenAI | 2 | **100%** | 96.2% | 100% |
| Gemini 2.5 Pro | Google | 3 | **100%** | 100% | 100% |
| Gemini 3.1 Pro Preview | Google | 1 | **100%** | 100% | 100% |
| Gemini 3.5 Flash | Google | 2 | **100%** | 100% | 100% |
| Gemini 2.5 Flash | Google | 4 | **95.0%** | 85.1% | 74.0% |
| Mistral Medium 3.5 | Mistral | 4 | **82.7%** | 76.9% | 82.0% |
| Mistral Large 3 | Mistral | 1 | 69.2% | 69.2% | 69.2% |
| GPT-4o-mini | OpenAI | 1 | **69.2%** | 69.2% | 61.5% |

Frontier models (Opus, Sonnet, Haiku, GPT-5.5, Gemini 2.5 Pro, Gemini 3.1 Pro, Gemini 3.5 Flash) achieve **100% GCF** on every run. GCF averages equal or better than JSON on every model. TOON is consistently the weakest format.

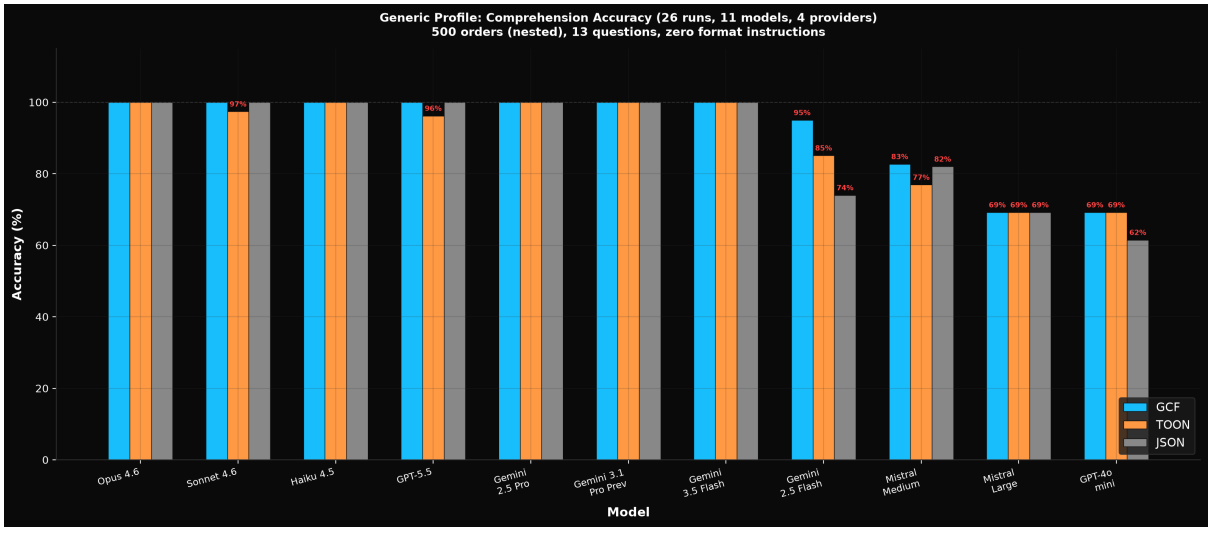


**Figure 6:** Generic Comprehension Accuracy

### 6.1b Graph Profile: Under Structural Stress

25 runs across 10 models. 500 symbols, 200 edges, 13 structured extraction questions with deterministic ground truth (no LLM judge). Each run generates a fresh random payload. Zero format instructions in the prompt.

| Model | Runs | GCF avg | TOON avg | JSON avg |
|---|---|---|---|---|
| Claude Opus 4.6 | 2 | **96.2%** | 88.5% | 73.1% |
| Claude Sonnet 4.6 | 2 | **100%** | 73.1% | 53.8% |
| Claude Haiku 4.5 | 2 | **96.2%** | 69.2% | 57.6% |
| GPT-5.5 | 5 | **84.1%** | 67.7% | 45.8% |
| GPT-5.4 | 4 | **78.0%** | 56.0% | 44.1% |
| GPT-5.4-mini | 2 | **71.8%** | 64.1% | 54.1% |
| Gemini 2.5 Pro | 2 | **100%** | 78.5% | 65.5% |
| Gemini 3.1 Pro | 1 | **100%** | 76.9% | 46.2% |
| Gemini 3.5 Flash | 1 | **100%** | 61.5% | 46.2% |
| Gemini 2.5 Flash | 4 | **85.5%** | 52.5% | 54.3% |

**GCF wins 24 of 25 runs (1 tie, 0 losses).** Five models achieve 100%.

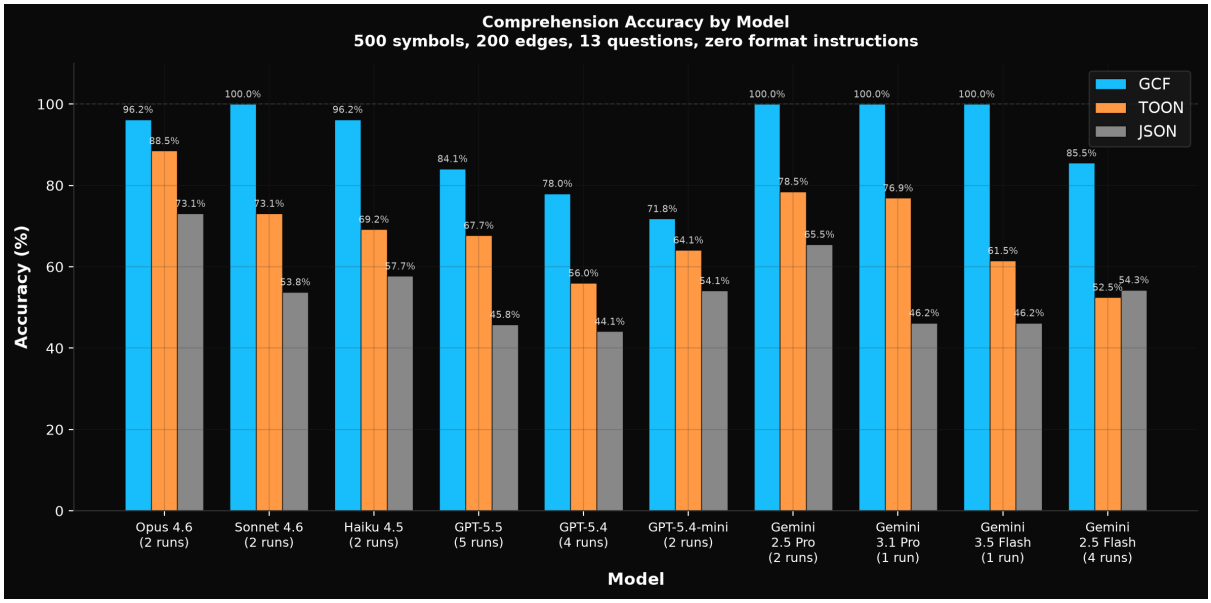


**Figure 7:** Comprehension Accuracy by Model

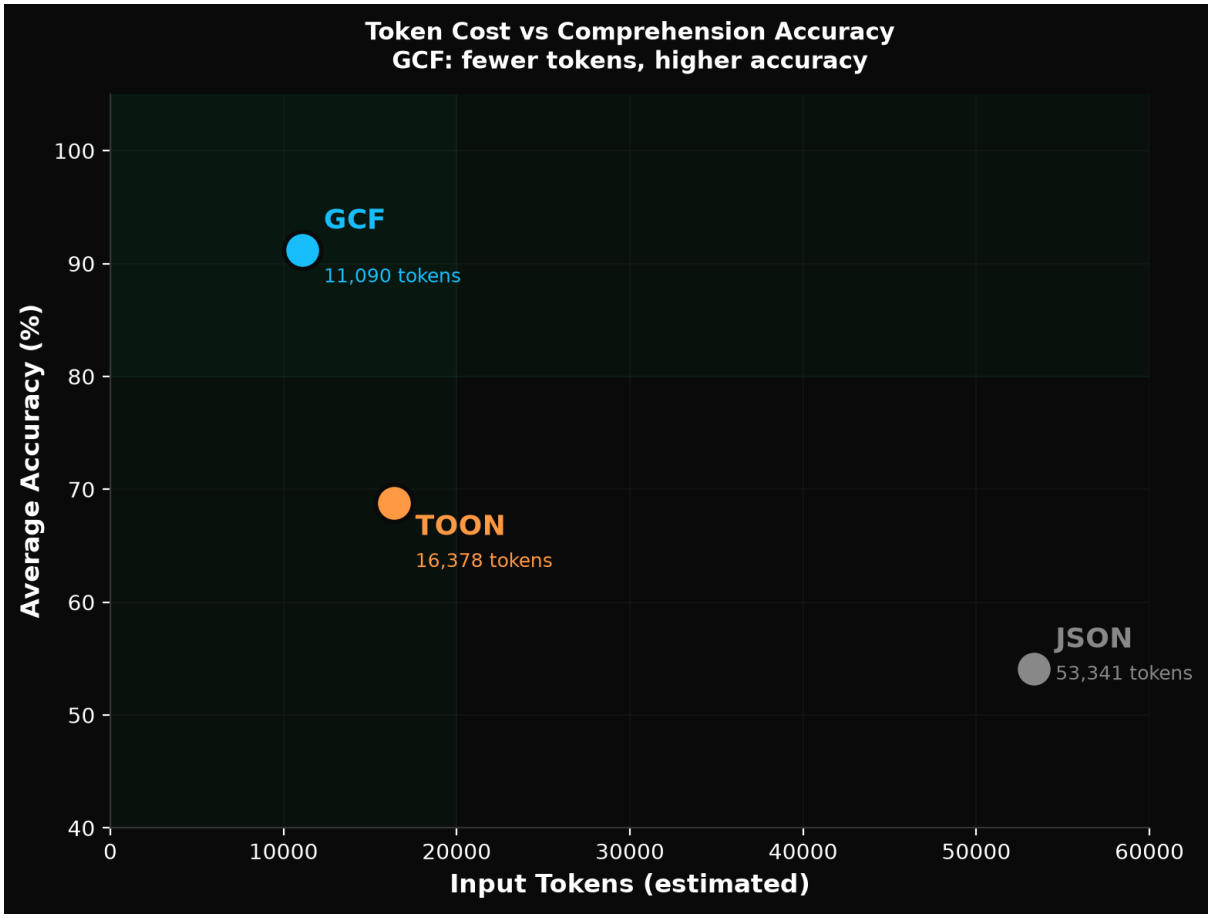


**Figure 8:** Token Cost vs Comprehension Accuracy

**Failure taxonomy** GCF, TOON, and JSON produce qualitatively different failure modes. The tokenizer analysis provides the mechanistic link: JSON's grammar symbols merge with field names in BPE tokenizer vocabularies, creating hidden structural boundaries that compound per row. Controlled experiments confirm this is causal: models trained with merge barriers develop 50-161 delimiter-specialized attention heads and achieve 3-738x lower structured data perplexity. Every head in a standard BPE model is structurally stranded by delimiter merging. See "Tokenizer-Attention Coupling" DOI: 10.5281/zenodo.20925910 for the full analysis.

**GCF fails on precision** (median error: 4). Off-by-1-2 header misreads (8 occurrences), deterministic column scan miscounts on GPT-5.4 (11), field confusion (2), miscellaneous (5), and context overwhelm empty responses on GPT-5.5 (10). The format structure is understood; the count is slightly misread. 36 total failures across 25 runs. GCF's pipe delimiter has a 0.47% merge rate across 43 tokenizers, meaning the model always sees clean structural boundaries.

**TOON fails on comprehension** (median error: 53). Distance grouping failures across all models (45 occurrences), column scan miscounts (10), attention decay on the last row (7), calls edge miscounts (10), symbol count wrong (2), and context overwhelm (20). The model cannot filter a flat 500-row table by column value. 94 total failures across 25 runs. TOON's tab delimiter has a 32.91% merge rate and 1,238 mergeable words in tokenizer vocabularies, the worst of any common separator character.

**JSON fails on structural overwhelm** (median error: 56). Empty string responses where the model produces nothing (33), massive undercounts (14), distance filter failures (44), column scan miscounts (37), and attention decay (3). At 53,000 tokens of repeated field names, the format itself prevents comprehension. JSON's grammar attention collapses from 30% to 8.6% at scale as the model stops attending to structural tokens. 131 total failures across 25 runs.

On two separate runs, Claude Opus responded to a JSON counting question by manually enumerating symbols one by one (143 lines on run 1, 119 on run 2), burning output tokens on a chain-of-thought enumeration and still getting the wrong answer. GCF answers the same question from a 3-character header: `[167]`.

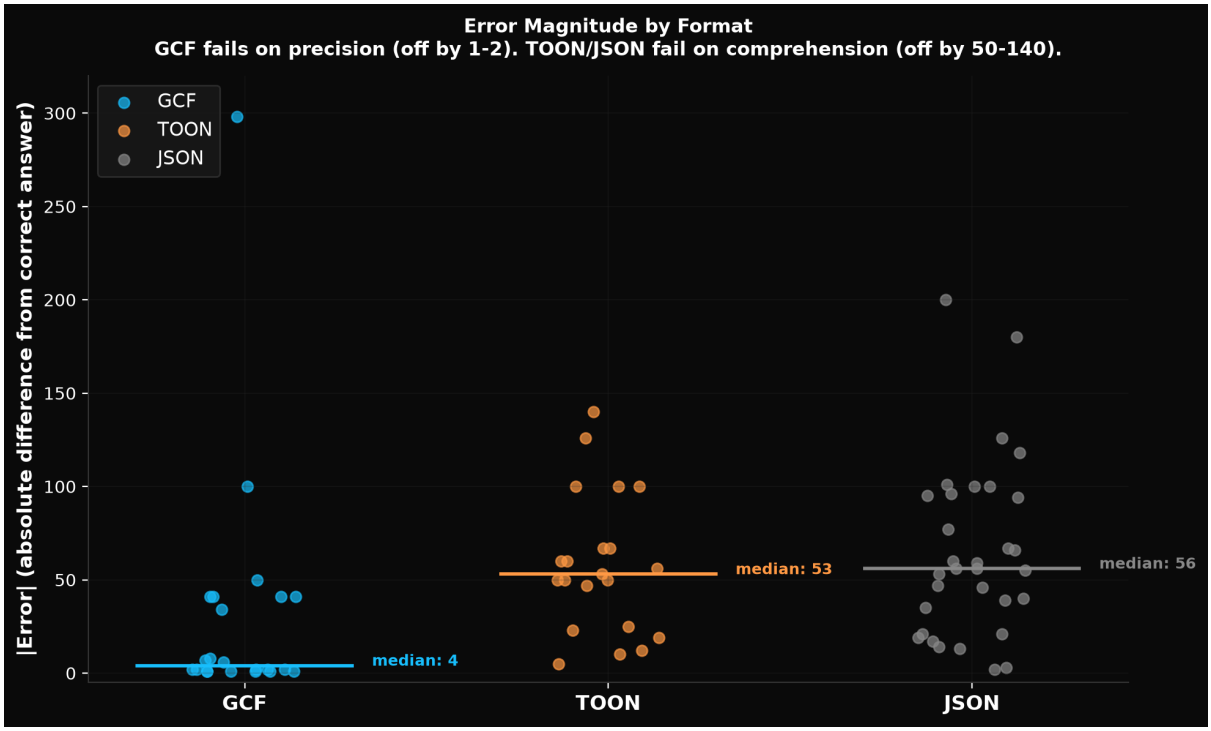


**Figure 9:** Error Magnitude by Format

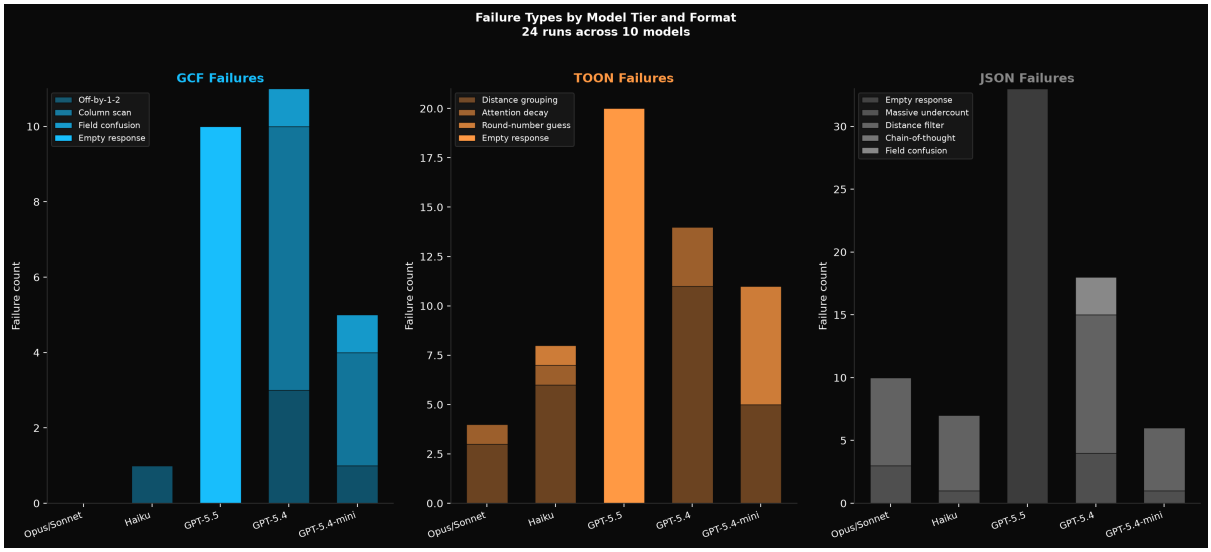


**Figure 10:** Failure Types by Model Tier

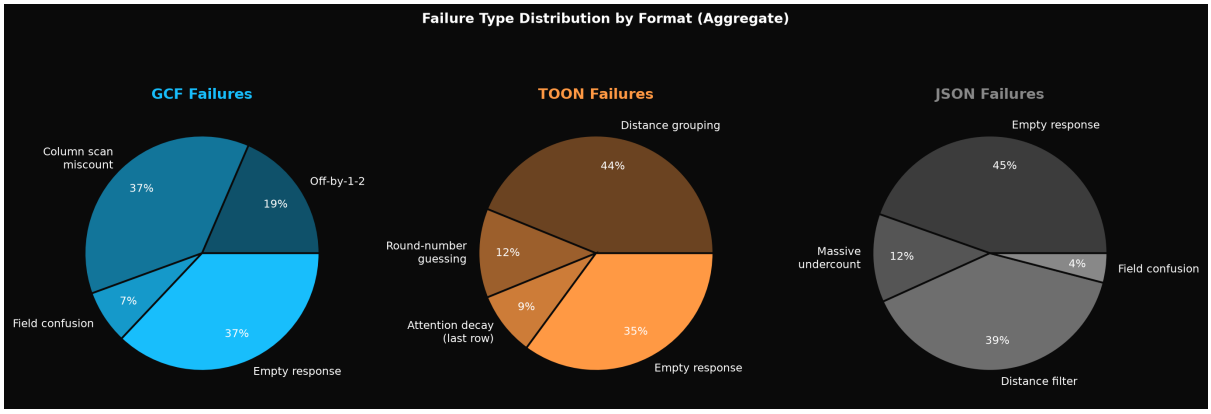


**Figure 11:** Failure Type Distribution

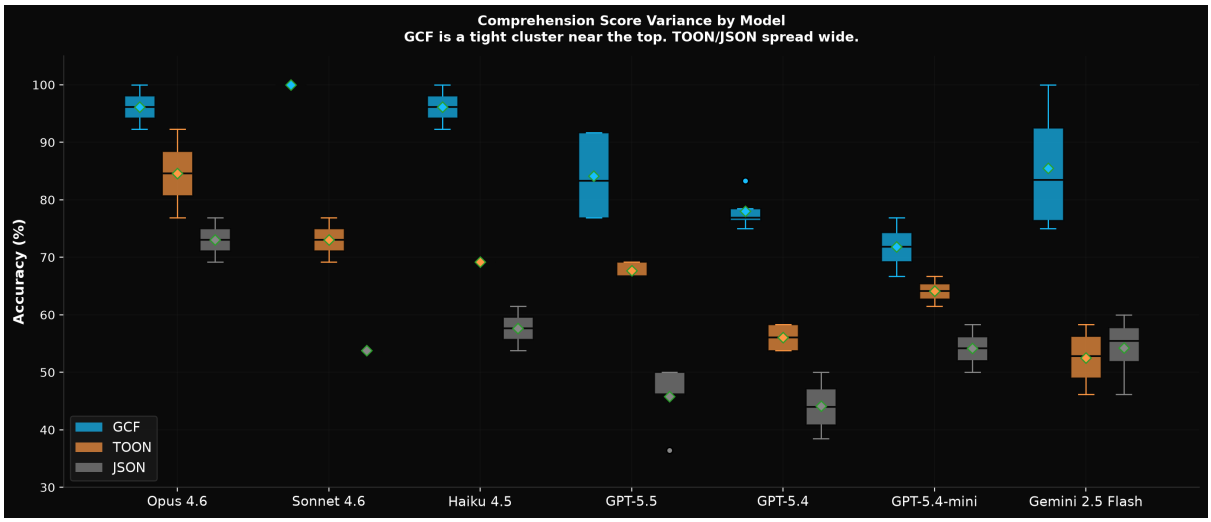


**Figure 12:** Comprehension Score Variance

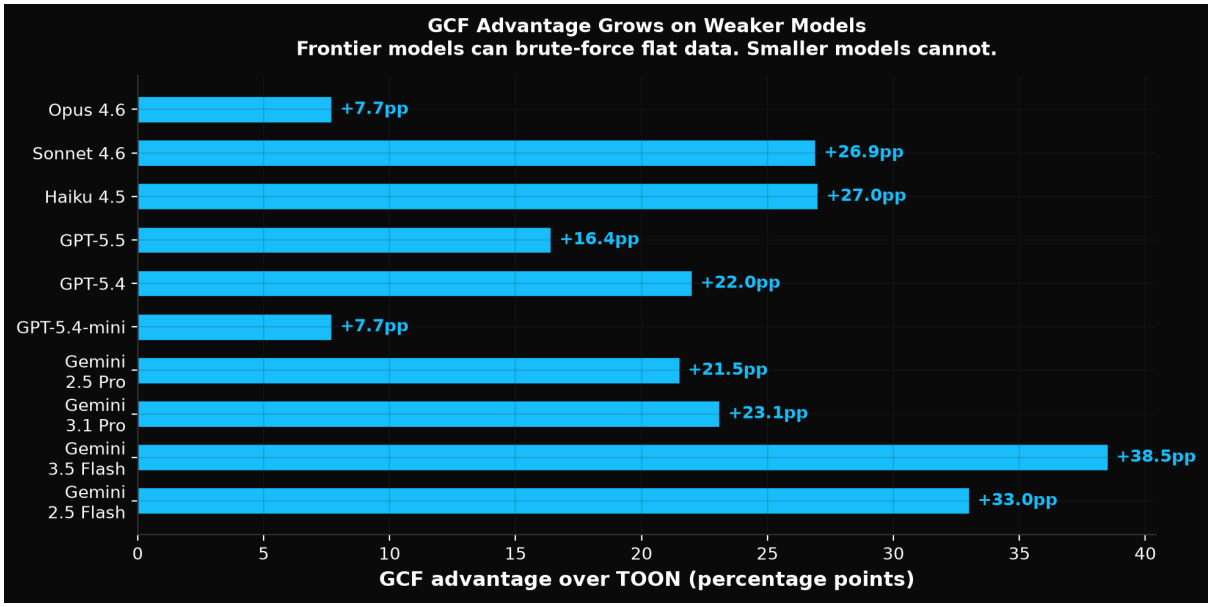


**Figure 13:** GCF Advantage Grows on Weaker Models

#### Comprehension methodology

- OpenAI runs used default temperature (non-zero). `EVAL_TEMPERATURE=0` available for deterministic runs.
- Claude evals via `claude -p` CLI with `--model` flag.
- OpenAI evals via chat completions API with exponential backoff on 429s.
- Google evals via generativelanguage API with retry logic.
- TOON encoding uses the official `toon-go` library from the `toon-format` GitHub organization.
- All raw logs in `eval/results/comprehension/`.

### 6.2 Session Statefulness Savings

| Call | New symbols | Reused symbols | GCF tokens | Cumulative savings vs JSON |
|---|---|---|---|---|
| 1 | 10 | 0 | 233 | 75.9% |
| 2 | 5 | 4 | 128 | 82.3% |
| 3 | 3 | 6 | 87 | 86.1% |
| 4 | 2 | 7 | 62 | 89.4% |
| 5 | 1 | 8 | 41 | 92.7% |

By the fifth tool call in a session, GCF achieves 92.7% token savings versus JSON because 8 of 9 referenced symbols are bare ID references (`@0`, `@3`) consuming 1 token each instead of 15-20 tokens for the full qualified name.

At production scale (500 symbols, 200 edges per call), session deduplication achieves 84.3% cumulative savings over 5 calls (148,360 JSON tokens vs 23,270 GCF tokens). Per bare reference: 2 tokens vs 19 tokens for a full declaration (89% savings per deduplicated symbol). JSON has no deduplication mechanism; every call retransmits the full payload.

**Production-scale savings curve (500 symbols, 200 edges):**

| Call # | JSON tokens | Session tokens | Session+Delta | Session/JSON | Stacked/JSON |
|---|---|---|---|---|---|
| 1 | 34,854 | 11,154 | 11,154 | 68.0% | 68.0% |
| 2 | 32,862 | 5,257 | 2,636 | 84.0% | 92.0% |
| 3 | 31,666 | 4,380 | 1,587 | 86.2% | 95.0% |
| 5 | 30,474 | 4,170 | 305 | 86.3% | 99.0% |
| 10 | 29,072 | 3,925 | 171 | 86.5% | 99.4% |

By call 10, each GCF response costs 171 tokens where JSON costs 29,072 (99.4% per-call savings).

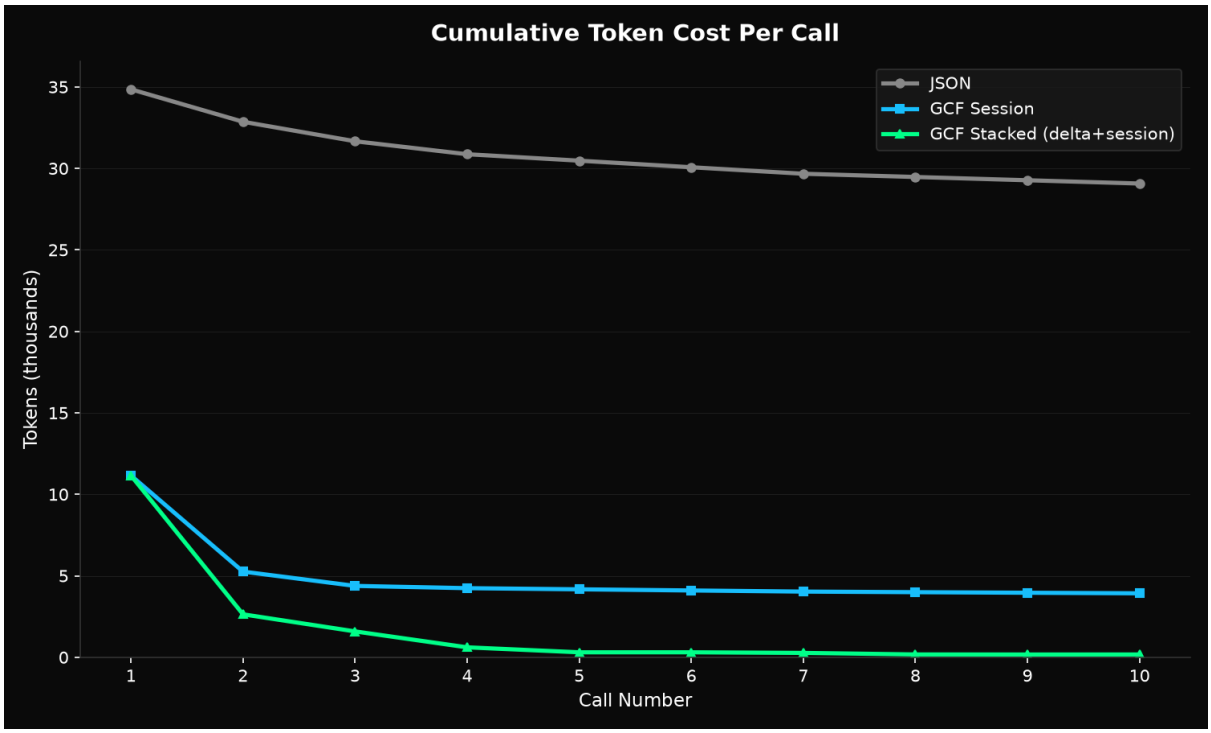


**Figure 14:** Session savings curve

The savings are structural and tokenizer-independent: validated across 8 production tokenizers (GPT-4o, Claude, LLaMA 3.1, Gemma 2, Mistral 7B, Qwen 2.5, DeepSeek V3, Phi-4) with a range of 87.2% to 89.5% (2.3pp spread).

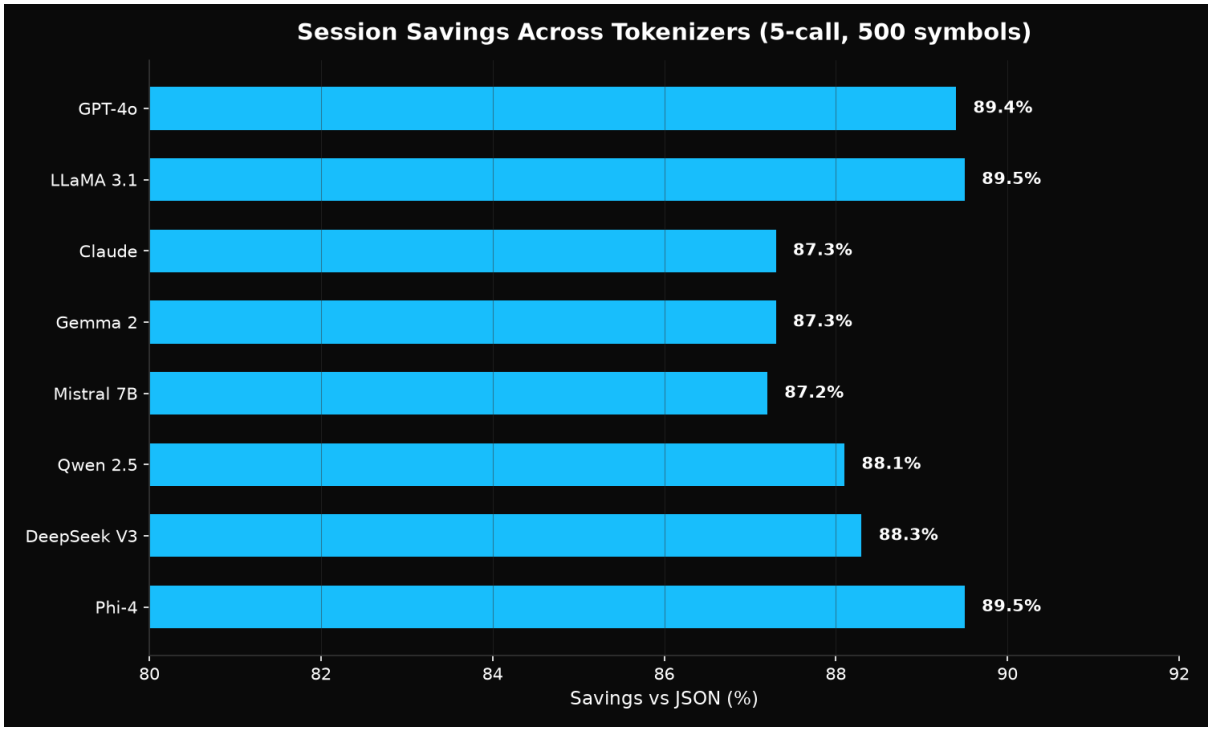


**Figure 15:** Cross-tokenizer session savings

**Comprehension validation:** Session dedup was validated on Gemini 2.5 Pro and Gemini 2.5 Flash. Attribute resolution (kind, provenance, score) through bare refs: 100% on both models. Zero degradation through 15 consecutive calls (31 messages). The LLM reads bare refs (@7) from prior responses in its context window with perfect accuracy. Session dedup matches full retransmission accuracy on every test.

---

## 7. Comparison to Alternatives

### 7.1 Columnar/TSV

Column headers with tab-separated values eliminate field name repetition, achieving ~27% token savings. But TSV treats graph data as flat tables: edge references still require full identifier strings in each row. TSV cannot exploit referential identity (local IDs) or topological encoding (edge arrows). GCF triples the savings.

### 7.2 Binary Formats (Protobuf, MessagePack, FlatBuffers)

These optimize for machine parsing and byte size. An LLM cannot read a protobuf payload; it must be decoded to text first, and the decoded text is typically JSON, eliminating the savings at the point of consumption. Binary formats solve a different problem (machine efficiency) than GCF (LLM token efficiency).

### 7.3 JSON-LD / RDF

Verbose by design: full URIs, type annotations, `@context` declarations. Optimizes for semantic interoperability across systems, not token-constrained consumption. JSON-LD is worse than JSON for LLM tool responses.

### 7.4 Markdown / Freeform Text

Some tools return markdown-formatted text. This is human-optimized, not LLM-optimized. Markdown uses whitespace for structure (indentation, line breaks, headers) which tokenizers handle inconsistently. GCF's positional format produces consistent, predictable token counts regardless of tokenizer implementation.

### 7.5 TOON (Token-Oriented Object Notation)

TOON is a tabular encoding format for JSON that declares array fields once and uses comma-separated rows. It achieves 30-60% savings versus JSON on flat tabular data.

**TOON:**

```
tool: example
symbols[3]{name,kind,score,distance}:
  pkg.Auth,function,0.9,0
  pkg.Server,function,0.7,1
  pkg.Cache,type,0.4,2
edges[1]{source,target,type}:
  pkg.Server,pkg.Auth,calls
```

**GCF (same data):**

```
GCF profile=graph tool=example symbols=3 edges=1
## targets
@0 fn pkg.Auth 0.90 lsp
## related
@1 fn pkg.Server 0.70 lsp
## extended
@2 type pkg.Cache 0.40 structural
## edges [1]
@0<@1 calls
```

The structural difference: TOON puts all symbols in one flat table with a `distance` column. GCF groups them into `## targets`, `## related`, `## extended` sections. This difference drives

both the comprehension gap (models can't filter flat tables at scale) and the generation gap (models write "target" instead of 0 in TOON's distance column).

**Methodology:** We forked TOON's benchmark repository (`github.com/blackwell-systems/toon-benchmark`), added GCF as one additional formatter, and expanded from their original 6 datasets to 16 representing real-world MCP tool responses. Tokenizer (o200k_base) and methodology are upstream. GCF wins 15 of 16, 29% fewer tokens overall, 54.8% fewer than JSON. TOON's single win (LSP symbols dataset, +77 tokens, 1.4%) is a tokenizer artifact where comma tokenizes marginally better than pipe on one flat tabular shape. GCF saves 218,480 tokens across the other 15 datasets. See benchmarks for the full table.

**Tokenization analysis:** TOON's tab delimiter merges with adjacent content more aggressively than JSON's quote character. Across 43 tokenizers and 29,025 checks: GCF pipe merge rate 0.47%, JSON quote merge rate 8.17%, TOON tab merge rate 32.91%. An exhaustive vocabulary scan reveals GPT-4 has 1,173 tab+letter vocabulary entries vs 22 pipe+letter entries. TOON's tab has 1,238 unique mergeable words across all 43 vocabularies, 52x the pipe's 24. Controlled experiments confirm the downstream effect: models trained with merge barriers achieve 2.3x better comprehension on tab-separated data never seen during training. TOON chose the delimiter with the largest adversarial surface of any common separator character. See tokenizer analysis and "Tokenizer-Attention Coupling" DOI: 10.5281/zenodo.20925910.

**GCF wins 15 of 16 datasets** (29% fewer tokens overall). TOON's one win is 77 tokens on a single dataset on edge cases (nested config and one tokenizer artifact).

TOON has no local-ID system, no edge encoding, no session deduplication, no delta encoding, and no streaming mode. These are structural limitations that cannot be added without a fundamental redesign. TOON edges must repeat the full qualified name of both source and target (~100 tokens per edge vs ~4 for GCF). TOON retransmits every record on every call; GCF tracks what's been sent. TOON's spec mandates upfront `[N]` counts with no deferred mechanism; GCF streams with zero buffering.

**TOON is fundamentally fragile for generation.** Its official decoder rejects LLM-generated output on 7 of 9 models tested. The failure is always the same: models write semantic labels where TOON expects integers. This is a structural design flaw in flat tabular formats that encode categories as column values. GCF solves this by expressing categories through section placement. See Section 9.1 for the full analysis.

On output, GCF produces 63% smaller output than JSON and 33% smaller than TOON at 100 symbols.

Fork with reproducible results: github.com/blackwell-systems/toon (branch: gcf-comparison).

### 7.6 Custom Compressed JSON

JSON with shortened field names (`"qn"` instead of `"qualified_name"`) achieves 15-25% savings. This preserves JSON's structural overhead (delimiters, quoting, nesting) while reducing readability. GCF eliminates the structural overhead entirely rather than shortening the labels on it.

---

## 8. Limitations and Validation

**LLM comprehension.** 2,500+ evaluations across 11 models and 4 providers validate this design. On standard workloads (500 orders, nested data), GCF achieves 100% on every frontier model. On structurally complex code graphs (500 symbols, 200 edges), GCF averages 90.7% where JSON drops to 54.1% and TOON to 68.8%. The concern that LLMs might struggle with GCF's dense positional format was unfounded; the format improves comprehension by eliminating both structural overhead and structural ambiguity. GCF's pipe delimiter maintains 99.5% grammar isolation across 43 tokenizers (0.47% merge rate), ensuring the model always sees clean structural boundaries regardless of which tokenizer it uses.

**LLM generation.** 28 generation runs across 9 models and 3 providers. GCF achieves 5/5 validity on every frontier model (Opus, Sonnet, GPT-5.5, Gemini 2.5 Pro, Gemini 3.1 Pro) with a 3-line primer and zero prior training. TOON's official decoder rejects LLM-generated output on 7 of 9 models. The failure is structural: TOON's flat columns encode semantic categories as integers, and models write labels ("target") instead of the expected integer (0). GCF expresses categories through section placement (`## targets`), aligning with how models naturally express grouped data. GCF output is 63% smaller than JSON and 33% smaller than TOON.

**Two profiles.** The graph profile is optimized for typed nodes and edges. The generic profile (`encodeGeneric`) handles arbitrary structured data (arrays of objects, nested records, mixed types). On TOON's own benchmark (expanded to 16 datasets), GCF wins 15 of 16 with 29% fewer tokens overall.

**Session statefulness requires coordination.** The session ID system assumes the server tracks which nodes have been transmitted to which client. Stateless deployments cannot use session compression. The non-session mode (full retransmission) remains available.

**Tokenizer dependency.** Token counts depend on the tokenizer. The benchmarks in this paper use o200k_base. Different tokenizers produce different token counts for the same GCF payload, though the relative savings versus JSON are consistent (within 2-3 percentage points across all 43 tested tokenizers). More importantly, GCF's structural boundaries are consistent: the pipe delimiter is always its own token on 43/43 tokenizers, while JSON's quote merges with field names on 13/43.

---

## 9. Implications for MCP and Agent Tooling

The MCP specification does not define a standard for tool response encoding beyond "the response is a JSON-RPC result." This means every MCP server independently decides how to format its output. The result is that agents receive verbose JSON from every tool, with no mechanism to request compact encoding.

We propose that MCP tool responses should support format negotiation: the client specifies a preferred encoding in the tool call, and the server returns the response in that encoding. GCF (or a format like it) should be available as a standard option alongside JSON.

The token savings are too large to ignore. A 50-92% reduction in tool response tokens (depending on data complexity and session reuse) translates directly to: lower API costs, faster time-to-first-token, more room in the context window for source code and reasoning, and fewer multi-turn loops caused by context window exhaustion.

### 9.1 LLM Generation: GCF as a Bidirectional Format

GCF is bidirectional: LLMs can produce it, not just consume it. 28 generation runs across 9 models, validated through real decoders (including TOON's official `toon-go` library).

| Model | GCF | TOON (natural) | JSON |
|---|---|---|---|
| Claude Opus 4.6 | **5/5** | 0/5 | 5/5 |
| Claude Sonnet 4.6 | **5/5** | 2-3/5 | 5/5 |
| Claude Haiku 4.5 | **5/5** | 1-3/5 | 5/5 |
| GPT-5.5 | **4-5/5** | 1-2/5 | 5/5 |
| GPT-5.4 | **5/5** | 0/5 | 5/5 |
| GPT-5.4-mini | **5/5** | 0/5 | 5/5 |
| Gemini 2.5 Pro | **5/5** | 1/5 | 5/5 |
| Gemini 3.1 Pro | **5/5** | 0/5 | 5/5 |
| Gemini 3.1 Flash Lite | **4-5/5** | 0/5 | 4-5/5 |

**GCF 5/5 on every frontier model. TOON fails on 7 of 9 models.**

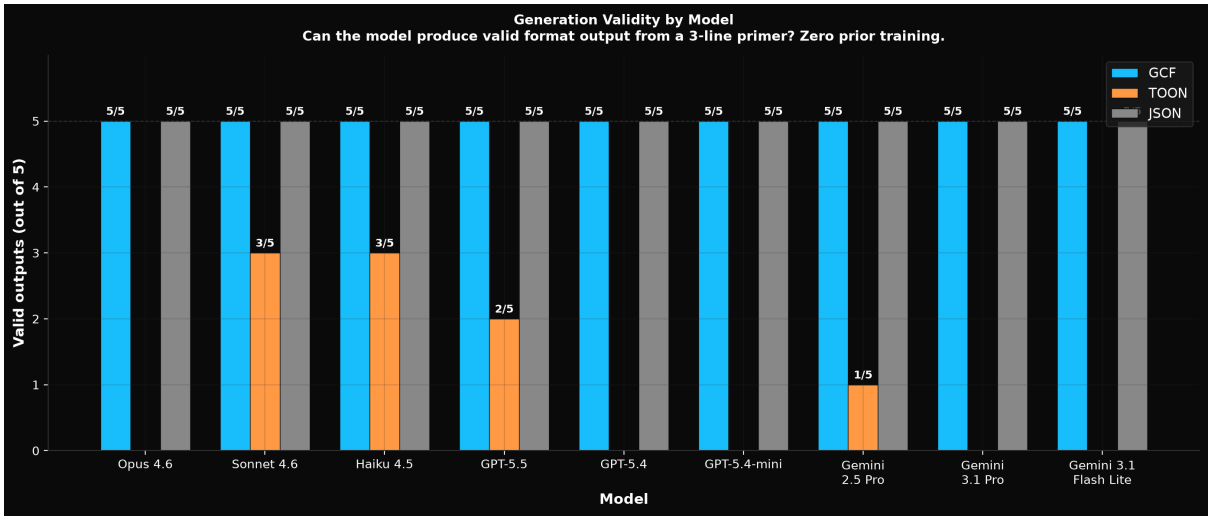


**Figure 16:** Generation Validity by Model

**Why TOON fails generation** TOON's flat tabular design encodes semantic categories as column values. When told "this symbol is a target," the model writes `target` in the distance column. TOON's

decoder expects `0`. The model would need to know, unprompted, that "target" maps to 0, "related" maps to 1, "extended" maps to 2. No model does this. The error is always the same: `toon: cannot assign string to int`.

This is a structural design flaw in flat tabular formats. Any time a column encodes a semantic category as an integer, the format is one prompt change away from producing invalid data. TOON is fundamentally fragile for LLM generation.

GCF expresses categories through section placement (`## targets`, `## related`). The model writes the symbol in the section matching the label. No integer mapping required. The format aligns with how LLMs naturally express grouped data.

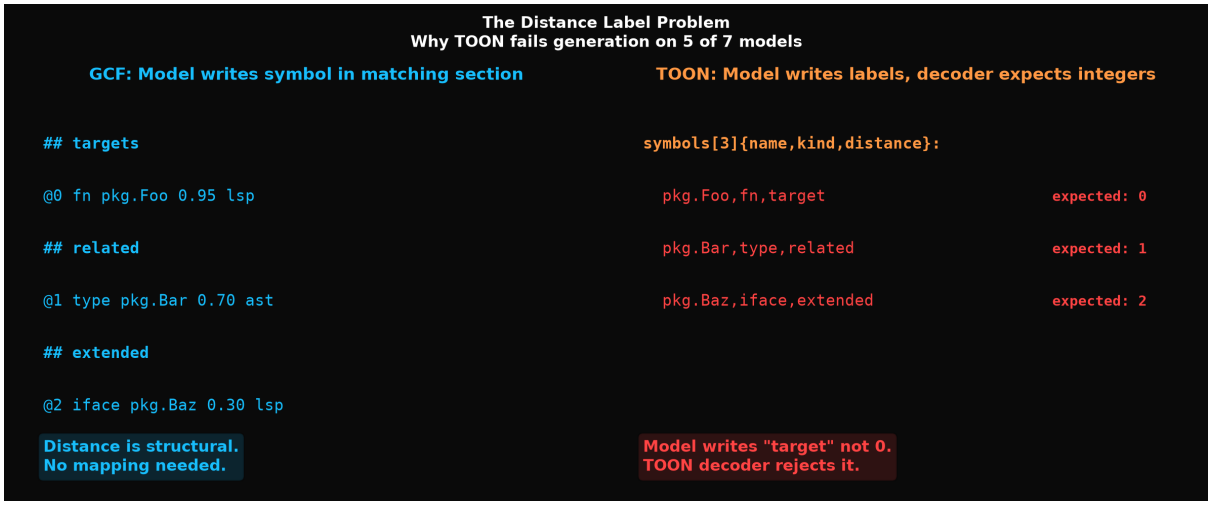


**Figure 17:** The Distance Label Problem

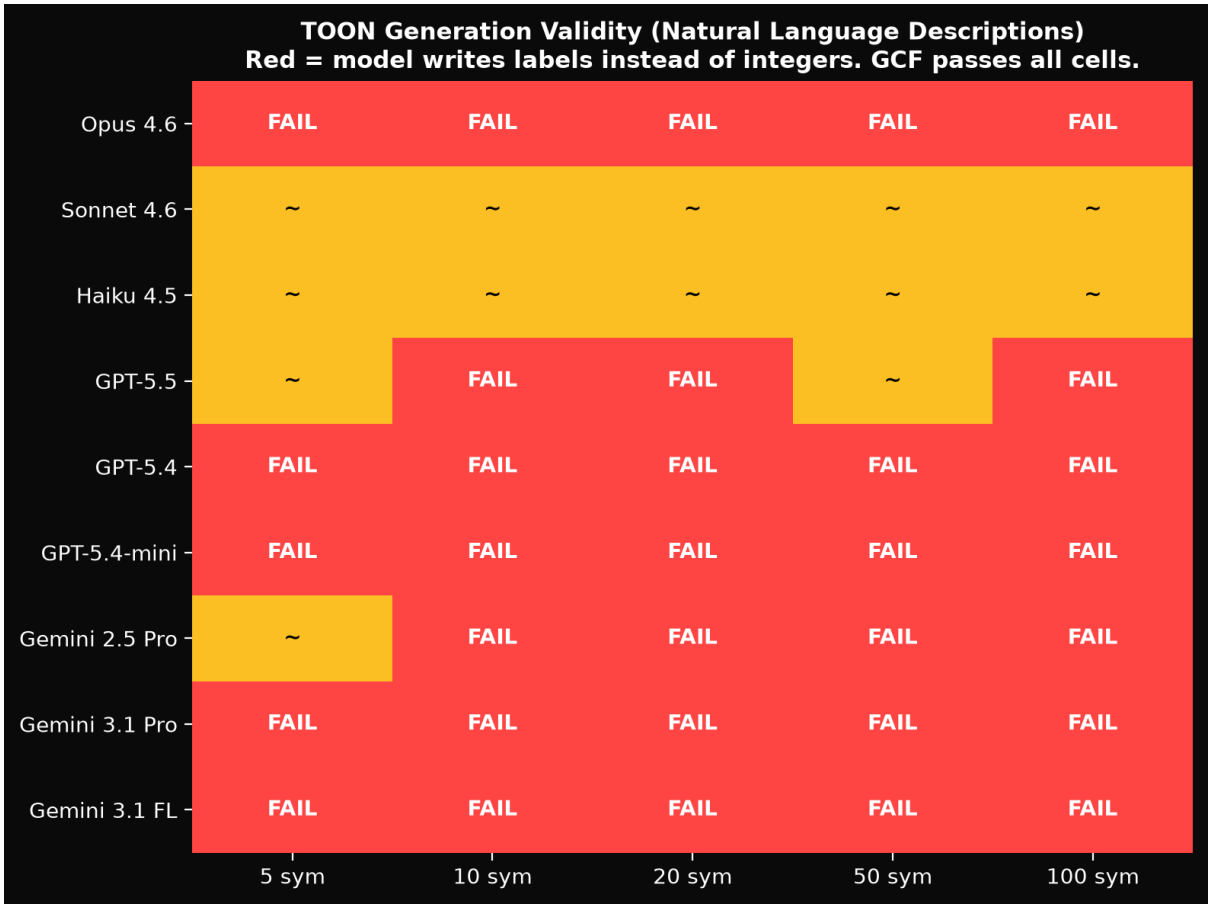


**Figure 18:** TOON Generation Heatmap

When TOON is given pre-encoded integers (hand-holding the model through the mapping), performance improves on some models but remains inconsistent. Even in the best case, GCF output is 28% smaller.

**Output size comparison**

| Format | 100 sym output | vs JSON |
|---|---|---|
| **GCF** | **5,976 B** | **63% fewer** |
| TOON | 8,937 B | 45% fewer |
| JSON | 16,121 B | baseline |

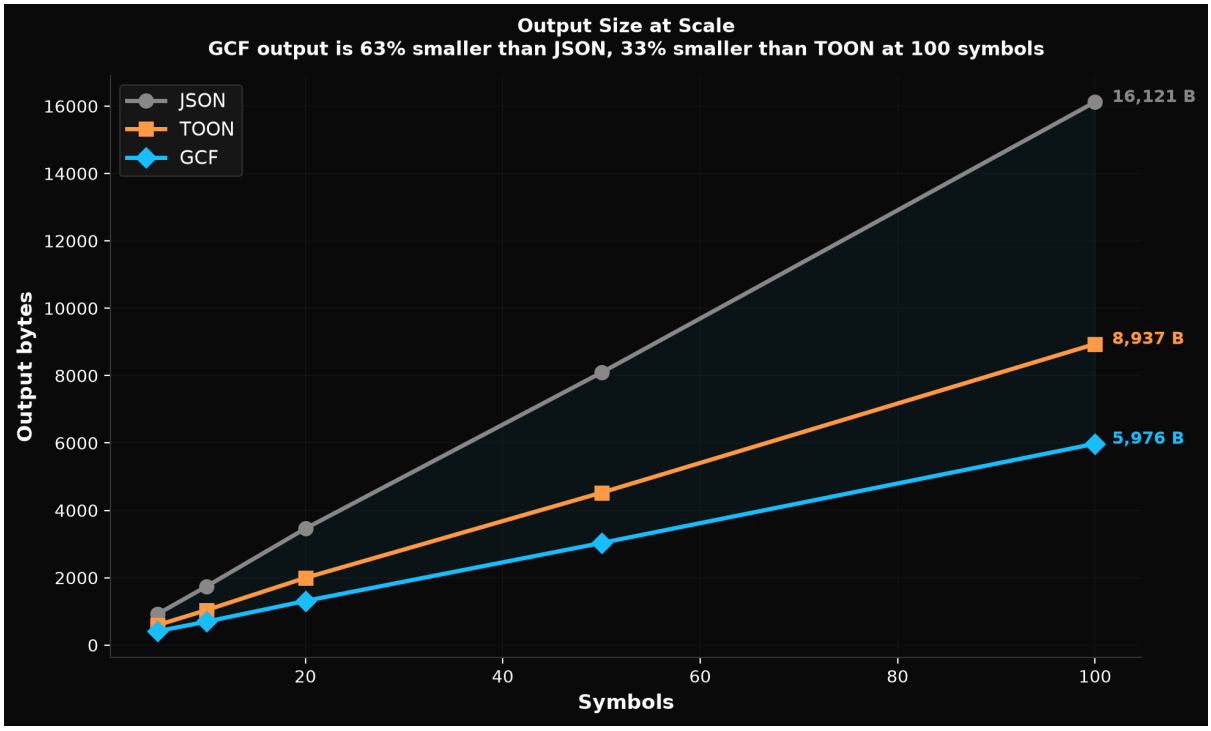


**Figure 19:** Output Size at Scale

**Implications.** GCF generation enables: (1) agent-to-agent communication at 63% fewer tokens per handoff, (2) structured output mode as an alternative to JSON mode, and (3) system prompt encoding where context packs are encoded as GCF to maximize context window utilization.

### 9.2 Streaming: Progressive Context Delivery

The streaming encoding extension (Section 3.8) enables a new interaction pattern: **progressive context delivery**. An MCP proxy can emit GCF fragments as MCP progress notifications while the upstream server is still processing. The LLM receives partial context immediately, reducing perceived latency from seconds to milliseconds on large graph traversals.

Combined with MCP's Streamable HTTP transport (Server-Sent Events), this creates a pipeline where graph data flows from the server through the proxy to the LLM token by token, with the trailer summary providing count verification at the end.

### 9.3 Delta Encoding

GCF's token savings compound with delta encoding: when the agent passes a `pack_root` from a prior call and the pack changed, the server sends only added/removed symbols instead of the full payload. Measured: 81.2% additional token savings at 96.6% symbol overlap on re-query scenarios. Combined with GCF's baseline savings and session deduplication, the three-level stack achieves over 97% cumulative token reduction on warm sessions versus stateless JSON.

## 10. Conclusion

JSON fails at scale for two compounding reasons. The visible reason is overhead: 81% of JSON's tokens are repeated field names and structural characters. The invisible reason is structural ambiguity: JSON's grammar characters merge with adjacent content in BPE tokenizer vocabularies, hiding field boundaries inside single tokens. At 500 rows, the model must process approximately 8,500 overhead tokens that are also structurally ambiguous. Grammar attention collapses from 30% to 8.6%. Comprehension drops to 54.1%.

GCF eliminates both problems. Header factoring eliminates the overhead (field names declared once, 97.7% signal at 500 rows vs JSON's 19%). Delimiter selection from the empirically verified low-merge character set eliminates the ambiguity (pipe: 0.47% merge rate, 24 mergeable words vs JSON's 1,939). Controlled experiments prove this is causal: models trained with merge barriers achieve 3-738x better structured data comprehension and 1.5x better code comprehension, with zero cost to natural language. The models develop 50-161 attention heads dedicated to parsing structure, while standard BPE models develop counterproductive heads that actively damage comprehension at scale.

GCF is bidirectional. 2,500+ LLM evaluations across 11 models and 4 providers prove it. Comprehension: 100% on every frontier model on standard workloads; 90.7% on structurally complex code graphs where JSON drops to 54.1% and TOON to 68.8%. Generation: 5/5 validity on every frontier model where TOON fails on 7 of 9 models. Output: 63% fewer tokens than JSON, 33% fewer than TOON. Session deduplication (84.3% cumulative savings over a 5-call session), delta encoding (81.2% on re-queries), and streaming encode (zero-buffering with trailer summary) compound savings across multi-turn interactions. No competing format offers these features.

The format is text-based, LLM-optimized, and implementable in any language. Implementations exist in six languages (Go, TypeScript, Python, Rust, Swift, Kotlin) with zero or minimal runtime dependencies. A drop-in MCP proxy enables adoption with zero code changes and adds streaming progress notifications for immediate partial context delivery.

GCF is a wire format. Wire formats are not optimized for human readability. HTTP headers are not readable. Protobuf is not readable. Nobody cares; they use a viewer. GCF is the wire format; JSON is the viewer format. The agent reads GCF (cheap, accurate), does its work, then calls `decode()` at the end if a human needs to see the result.

The format that looks clean to humans (JSON) is the one that breaks for agents at scale: its overhead wastes tokens and its grammar hides structural boundaries. The companion paper ("Tokenizer-Attention Coupling") quantifies the hidden cost: every attention head in a standard BPE model has 4x more structural capacity than the tokenizer allows it to use, and this constraint is permanent from step 5,000 of training onward. Standard BPE models at 1.3B scale develop 124 counterproductive delimiter heads whose removal improves comprehension by 57%. The tokenizer is not a preprocessing step; it is an architectural constraint that permanently shapes what the model can do with structured input.

GCF is, to our knowledge, the only wire format whose grammar was reverse-engineered from this kind of attention-level analysis. Every design choice (pipe delimiters, header factoring, positional encoding) was informed by empirical measurement of how tokenizers and attention mechanisms interact.

The result: 100% accuracy on every frontier model for standard workloads and 90.7% on structurally complex data, at the lowest token cost, with grammar that never merges with field names on any tested tokenizer. This is not a tradeoff. It is a design methodology validated by 2,500+ evaluations, a 43-tokenizer vocabulary analysis, controlled training experiments across two architectures and three domains, and 12 production deployments.

---

### Reference Implementation

- **Specification:** SPEC.md (v3.2 Stable, RFC 2119 keywords, conformance checklists, streaming extension, error taxonomy). Published at gcformat.com.
- **Go library:** `github.com/blackwell-systems/gcf-go` (v0.6.1)
- **TypeScript library:** `@blackwell-systems/gcf` on npm (v0.6.1)
- **Python library:** `gcf-python` on PyPI (v0.5.1)
- **Rust library:** `gcf` on crates.io (v0.5.1)
- **Swift library:** `gcf-swift` via SPM (v0.5.1)
- **Kotlin library:** `gcf-kotlin` via JitPack (v0.5.1)
- **MCP proxy:** `github.com/blackwell-systems/gcf-proxy`: streaming progress notifications, drop-in wrapper, zero code changes
- **Comprehension eval:** `github.com/blackwell-systems/gcf-go/eval` (generic profile: 500 orders, 27 runs, 11 models; graph profile: 500 symbols, 24 runs, 10 models; 13 questions, 3 formats)
- **Tokenizer analysis:** `github.com/blackwell-systems/gcf/eval` (13 scripts, 43 tokenizers, 20 providers). See "Tokenizer-Attention Coupling" DOI: 10.5281/zenodo.20925910.
- **Generation eval:** `github.com/blackwell-systems/gcf-go/eval` (5-100 symbols, GCF vs TOON vs JSON, 9 models, validated through real decoders)
- **Eval results:** `github.com/blackwell-systems/gcf/eval/results` (all raw logs, failure taxonomy, artifacts)
- **TOON benchmark fork:** `github.com/blackwell-systems/toon` (branch: gcf-comparison, their datasets, their tokenizer)
- **Conformance test suite:** `github.com/blackwell-systems/gcf/tests/conformance` (157 v3 fixtures across both profiles, streaming, and normative errors)
- **Interactive playground:** gcformat.com/playground (three-way JSON vs TOON vs GCF comparison using real @toon-format/toon library)
- **Production deployment:** 12 adopters including OmniRoute (6.1K stars), NetClaw (556 stars), NeuroNest (commercial), ctx (510 stars), Speakeasy, knowing (28 MCP tools), agent-lsp (66 MCP tools). Full list: gcformat.com/ecosystem/adopters

---

### Appendix A: Full Example

**JSON (965 tokens):**

```
{
  "tool": "context_for_task",
  "tokens_used": 1847,
  "token_budget": 5000,
  "symbols": [
    {
      "qualified_name":
       ↪  "github.com/blackwell-systems/knowing/internal/mcp.requireHash",
      "kind": "function",
      "score": 0.78,
      "signature": "func requireHash(args map[string]any, key string)
       ↪  (types.Hash, error)",
      "provenance": "lsp_resolved",
      "distance": 0,
      "components": { "blast_radius": 0.40, "confidence": 0.25, "recency":
       ↪  0.06, "distance": 0.15 }
    },
    {
      "qualified_name":
       ↪  "github.com/blackwell-systems/knowing/internal/mcp.NewServer",
      "kind": "function",
      "score": 0.54,
      "provenance": "lsp_resolved",
      "distance": 1
    }
  ],
  "edges": [
    {
      "source":
       ↪  "github.com/blackwell-systems/knowing/internal/mcp.NewServer",
      "target":
       ↪  "github.com/blackwell-systems/knowing/internal/mcp.requireHash",
      "edge_type": "calls"
    }
  ]
}
```

**GCF (233 tokens):**

```
GCF profile=graph tool=context_for_task budget=5000 tokens=1847 symbols=2 edges=1
## targets
@0 fn github.com/blackwell-systems/knowing/internal/mcp.requireHash 0.78 lsp_reso
## related
@4 fn github.com/blackwell-systems/knowing/internal/mcp.NewServer 0.54 lsp_resolv
## edges [1]
```

```
@0<@4 calls
```

Same semantic content. 75.9% fewer tokens.

---

### Appendix B: Streaming Example

**Buffered mode** (full payload known upfront):

```
GCF profile=graph tool=context_for_task budget=5000 symbols=3 edges=2
## targets
@0 fn pkg.Auth 0.95 lsp_resolved
@1 fn pkg.Handler 0.88 lsp_resolved
## related
@2 type pkg.Config 0.72 ast_inferred
## edges [2]
@0<@1 calls
@2<@0 references
```

**Streaming mode** (data arriving incrementally):

```
GCF profile=graph tool=context_for_task budget=5000
## targets
@0 fn pkg.Auth 0.95 lsp_resolved
@1 fn pkg.Handler 0.88 lsp_resolved
## related
@2 type pkg.Config 0.72 ast_inferred
## edges [?]
@0<@1 calls
@2<@0 references
##! summary symbols=3 edges=2 sections=targets:2,related:1,edges:2
```

Both produce identical `Payload` structures when decoded. The streaming mode enables zero-buffering encode with O(1) memory per row.

---